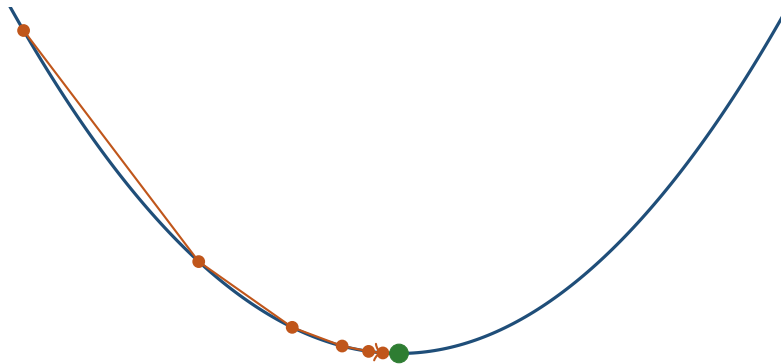


CLASE MAGISTRAL

Matemáticas para la Inteligencia Artificial

Del álgebra lineal al cálculo: cómo las matemáticas construyen y entrenan a la IA



Parte I. Álgebra lineal

Vectores y matrices

Transformaciones lineales y redes neuronales

Dependencia lineal en datasets

Parte II. Cálculo para la IA

Funciones, dominio, imagen y continuidad

Límites, derivadas y descenso de gradiente

Integrales: acumulación, promedios y AUC

Estudio completo de funciones de pérdida

Parte III. Álgebra lineal avanzada

Espacios vectoriales y producto punto

Núcleo, kernels y determinante

Matrices simétricas, positivas, ortogonales y dispersas

Teorema espectral y PCA

Índice

I Álgebra lineal: el lenguaje de los datos	2
1 Introducción y bienvenida	2
2 Los ladrillos de la IA: vectores	3
3 La estructura de los datos: matrices	4
4 Matrices como transformaciones lineales (el corazón de la IA)	5
5 Caso de estudio: matriz de 4×7 y dependencia lineal	7
Cierre de la Unidad 1	11
II Cálculo para la IA: cómo aprenden las máquinas	12
6 Unidad 2: Funciones en la Inteligencia Artificial	12
7 Unidad 3: Límites y derivadas, la brújula del aprendizaje	18
8 Unidad 4: Integrales, acumular y promediar errores	25
9 Unidad 5: Estudio completo de una función de pérdida	31
III Álgebra lineal avanzada: la geometría profunda de los modelos	38
10 Unidad 6: Espacios, núcleo y determinante	38
11 Unidad 7: Matrices con personalidad y el teorema espectral	46
Cierre general del curso	55

PARTE I

Álgebra lineal: el lenguaje de los datos

Unidad 1: Vectores, matrices y el lenguaje secreto de las máquinas

1. Introducción y bienvenida

¡Bienvenidos y bienvenidas!

Antes de empezar quiero decirles algo importante: **si nunca vieron álgebra lineal, están exactamente donde tienen que estar**. Esta clase está pensada para ustedes. No vamos a llenar el pizarrón de símbolos raros. Vamos a construir intuición, que es lo que de verdad importa para entender la IA.

La IA no es magia

Cuando ChatGPT responde una pregunta, cuando Netflix les recomienda una serie o cuando el celular reconoce su cara, parece que hubiera algo mágico, casi humano, detrás. La verdad es menos mística y mucho más interesante:

- **Toda la información que procesa una IA se convierte en números.** Una foto, una palabra, una canción o un usuario terminan siendo listas de números.
- **Esos números se organizan en estructuras llamadas vectores y matrices.**
- **La IA “piensa” multiplicando y sumando** esas estructuras, millones de veces por segundo.
- **“Aprender” significa ajustar números** dentro de esas matrices hasta que los resultados sean buenos.

En otras palabras: **la Inteligencia Artificial es álgebra lineal en acción**. Si la programación es el cuerpo de la IA, el álgebra lineal es su sistema nervioso.

¿Por qué es indispensable dominarla?

- Les permite **entender** qué hace un modelo, en vez de usarlo como una caja negra.
- Les permite **diagnosticar** problemas: por qué un modelo no aprende o por qué da resultados inestables.
- Es el **idioma común** de todos los papers, librerías (NumPy, PyTorch, TensorFlow) y profesionales del área.

Idea para llevarse

No necesitan ser matemáticos brillantes. Necesitan entender *qué representa* cada objeto matemático y *qué le hace* a los datos. Ese es nuestro objetivo de hoy.

2. Los ladrillos de la IA: vectores

¿Qué es un vector? Dos miradas complementarias

Mirada matemática: una lista ordenada de números.

$$\vec{v} = [3, 5, 2]$$

Dos palabras clave:

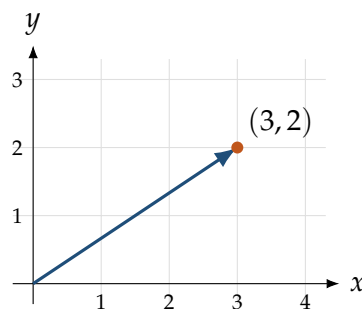
- **Lista:** son varios números juntos.
- **Ordenada:** el orden importa. $[3, 5]$ **no** es lo mismo que $[5, 3]$, porque cada posición significa algo distinto.

La cantidad de números que tiene un vector se llama su **dimensión**. El vector de arriba tiene dimensión 3.

Mirada geométrica: un punto o una flecha en el espacio.

Imaginen un plano con un eje horizontal y uno vertical, como un mapa. El vector $[3, 2]$ se puede ver como:

- Un **punto** ubicado 3 pasos a la derecha y 2 hacia arriba, o
- Una **flecha** que sale del origen $(0, 0)$ y llega a ese punto.



Con 3 números estamos en el espacio 3D, como una habitación. ¿Y con 100 números? No lo podemos dibujar, pero **la matemática funciona exactamente igual**. La IA trabaja cómodamente en espacios de cientos o miles de dimensiones.

¿Qué representa un vector en IA?

Un vector es la forma en que la IA **describe una cosa usando números**. Cada posición es una **característica** (en inglés, *feature*).

Ejemplo 1: una casa

Supongamos que queremos que una IA estime precios de casas. Describimos cada casa así:

$$\text{Casa} = \left[\underbrace{120}_{\text{m}^2}, \underbrace{3}_{\text{habitaciones}}, \underbrace{15}_{\text{años de antigüedad}} \right]$$

Para la IA, esa casa es ese vector. No ve ladrillos ni ventanas, ve un punto en un espacio de 3 dimensiones.

Ejemplo 2: un usuario en un sistema de recomendación

Una plataforma de streaming podría describir a un usuario según cuánto le gusta cada género (de 0 a 1):

$$\text{Ana} = \left[\underbrace{0.9}_{\text{acción}}, \underbrace{0.1}_{\text{romance}}, \underbrace{0.8}_{\text{ciencia ficción}} \right] \quad \text{Bruno} = [0.8, 0.2, 0.9]$$

La magia geométrica: Ana y Bruno son dos flechas que **apuntan casi en la misma dirección**. Para la IA, eso significa que **tienen gustos parecidos**. Así que si a Bruno le encantó una película, probablemente a Ana también le guste. **Así funcionan, en esencia, los sistemas de recomendación: midiendo qué tan cerca o alineados están los vectores.**

Idea para llevarse

En IA, **todo es un vector**: una casa, un usuario, una palabra, un píxel, un sonido. Y **cosas parecidas quedan cerca en el espacio**.

3. La estructura de los datos: matrices

¿Qué es una matriz?

Una **matriz** es una **tabla rectangular de números**, organizada en **filas** (horizontales) y **columnas** (verticales).

$$A = \begin{bmatrix} 120 & 3 & 15 \\ 80 & 2 & 30 \\ 200 & 5 & 2 \end{bmatrix}$$

Decimos que esta matriz es de 3×3 (3 filas por 3 columnas). **Siempre se nombra primero la cantidad de filas y después la de columnas.**

¿Cómo se conecta con los vectores?

Una matriz es simplemente **un montón de vectores apilados**:

- Cada **fila** es un vector.
- Cada **columna** también es un vector.

Es como una planilla de Excel: cada renglón es un vector y cada columna es otro vector.

La matriz en IA: el Dataset

Acá viene la conexión fundamental. En IA, **una matriz suele ser el conjunto de datos (dataset)** con el que entrenamos al modelo:

- **Cada fila = un ejemplo** (una casa, un usuario, una imagen, un paciente).
- **Cada columna = una característica** (metros cuadrados, edad, color de un píxel).

	m ²	Habitaciones	Antigüedad
Casa 1	120	3	15
Casa 2	80	2	30
Casa 3	200	5	2

Leída **por filas**, la tabla nos cuenta la historia de cada casa. Leída **por columnas**, nos muestra cómo varía una característica a lo largo de todas las casas (por ejemplo, la columna “m²” nos dice qué tan grandes son las casas del dataset).

¿Y una imagen? Una foto en blanco y negro de 28×28 píxeles es una matriz donde cada número indica qué tan claro u oscuro es cada píxel. Si “aplanamos” esa foto en un vector de 784 números, podemos apilar miles de fotos como filas de una matriz gigante: ese es el dataset de un reconocedor de imágenes.

Idea para llevarse

Un vector describe **una cosa**. Una matriz describe **muchas cosas a la vez**. El dataset es la materia prima de la IA, y tiene forma de matriz.

4. Matrices como transformaciones lineales (el corazón de la IA)

Hasta ahora vimos la matriz como una tabla que **guarda** datos. Ahora vamos a descubrir su segunda personalidad, la más poderosa: **la matriz como máquina que transforma vectores**.

La idea: una máquina de entrada y salida



Cuando **multiplicamos una matriz por un vector**, obtenemos un **nuevo vector**. Geométricamente, la matriz **agarra la flecha y la modifica**: la puede **estirar, encoger, rotar, reflejar o inclinar**.

A esto se lo llama **transformación lineal**. “Lineal” significa, en términos simples, que la transformación es “prolija”: **las líneas rectas siguen siendo rectas, las cuadrículas siguen siendo cuadrículas (aunque deformadas) y el origen no se mueve**.

¿Cómo se multiplica una matriz por un vector?

Cada fila de la matriz se “cruza” con el vector: se multiplican los números posición por posición y se suman.

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} a \cdot x + b \cdot y \\ c \cdot x + d \cdot y \end{bmatrix}$$

Ejemplos visuales

Estirar (cambiar de tamaño):

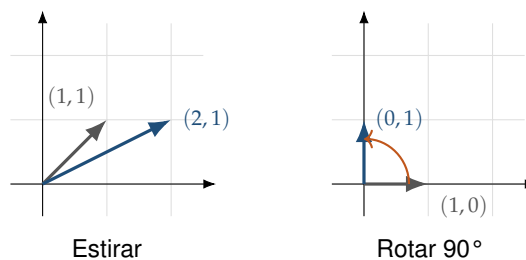
$$\begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \cdot 1 + 0 \cdot 1 \\ 0 \cdot 1 + 1 \cdot 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

⇒ Esta matriz **estira todo al doble en horizontal** y deja lo vertical igual. Es como agarrar una goma elástica de los costados.

Rotar:

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

⇒ La flecha que apuntaba a la derecha ahora apunta hacia arriba: **giró 90°**.



Imaginen que no transforman una sola flecha, sino **todo el espacio a la vez**, como si el plano fuera una hoja de goma cuadriculada que la matriz estira y gira. Esa es la visión geométrica que tienen que llevarse.

La conexión con las Redes Neuronales

Ahora sí, el momento clave de la clase.

Una capa de una red neuronal es, esencialmente, esto:

$$\text{salida} = f(W \cdot X + b)$$

Traduzcamos cada pieza:

- X es el **vector de entrada** (por ejemplo, los datos de una casa).
- W es la **matriz de pesos** (*weights*). Es la “máquina” que transforma el vector.
- b es el **sesgo** (*bias*), un pequeño vector que se suma para desplazar el resultado (es un ajuste fino).
- f es la **función de activación**, un pequeño “filtro” no lineal (por ejemplo, **ReLU**: deja pasar los positivos y convierte los negativos en 0).

Ejemplo concreto

Entrada: una casa descrita por $X = [120, 3, 15]$ (m^2 , habitaciones, antigüedad). Una capa con 2 neuronas tiene una matriz de pesos de 2×3 :

$$W = \begin{bmatrix} 0.01 & 0.5 & -0.1 \\ 0.02 & -0.2 & 0.05 \end{bmatrix}$$

$$W \cdot X = \begin{bmatrix} 0.01 \cdot 120 + 0.5 \cdot 3 + (-0.1) \cdot 15 \\ 0.02 \cdot 120 + (-0.2) \cdot 3 + 0.05 \cdot 15 \end{bmatrix} = \begin{bmatrix} 1.2 \\ 2.55 \end{bmatrix}$$

Aplicamos ReLU (ambos son positivos, así que quedan igual): la salida es $[1.2, 2.55]$.

¿Qué pasó? La capa tomó una casa descrita con 3 números y la **transformó** en una nueva representación de 2 números. Cada neurona combinó las características a su manera: la primera, por ejemplo, “valora” las habitaciones y “castiga” la antigüedad. Esas nuevas representaciones alimentan la capa siguiente, y así sucesivamente.

¿Y por qué hace falta la activación?

Porque **encadenar transformaciones lineales da otra transformación lineal**. Sin la activación, una red de 100 capas equivaldría a una sola matriz. La activación introduce “curvas” y permite que la red aprenda patrones complejos. **La matriz transforma; la activación dobla. Juntas, construyen inteligencia.**

Entonces, ¿qué significa “entrenar” una IA?

Al principio, los números de W son **aleatorios**: la transformación es un desastre y las predicciones son malas. Entrenar consiste en:

1. Pasar datos por la red y ver qué predice.
2. Medir **cuánto se equivocó** (el error).
3. **Ajustar levemente los números de W** para que el error baje.
4. Repetir miles o millones de veces.

Idea para llevarse

Entrenar una red neuronal es buscar la matriz correcta. Es encontrar la transformación que lleva los datos de entrada (“esta foto”) a la salida deseada (“es un gato”). Cuando escuchen que un modelo tiene “175 mil millones de parámetros”, piensen: *son los números de sus matrices de pesos*.

5. Caso de estudio: matriz de 4×7 y dependencia lineal

Ahora vamos a ponernos el guardapolvo de científicos de datos y analizar un dataset real (bueno, casi real).

El escenario

Una inmobiliaria de nuestra ciudad nos pasa un dataset para entrenar un modelo que estime precios. Tiene **4 casas** y **7 características**:

	C1 m ² cubiertos	C2 m ² jardín	C3 m ² totales	C4 Habitaciones	C5 Baños	C6 Antigüedad	C7 Km al centro
Casa A	80	20	100	2	1	10	5
Casa B	120	30	150	3	2	5	8
Casa C	60	0	60	1	1	30	2
Casa D	80	20	100	2	1	10	5

¿Qué significan las filas y las columnas acá?

- Las **4 filas** son **4 ejemplos** (4 casas). Cada fila es un vector de 7 números: la “ficha” completa de una casa. Geométricamente, cada casa es **un punto en un espacio de 7 dimensiones**.
- Las **7 columnas** son **7 características**. Cada columna es un vector de 4 números: muestra cómo varía esa característica entre las casas.

Así que esta matriz es de 4×7 : 4 filas, 7 columnas.

¿Ven algo sospechoso en la tabla? Tómense un momento antes de seguir leyendo.

¿Qué es la Dependencia Lineal (en términos simples)?

Un conjunto de vectores es **linealmente dependiente** cuando **al menos uno de ellos se puede fabricar combinando los otros** (multiplicándolos por números y sumándolos).

Analogía de la receta: Imaginen que tienen pintura roja, pintura azul y pintura violeta. Si el violeta se obtiene mezclando rojo y azul, entonces **el violeta es redundante**: no les da ningún color nuevo que no pudieran obtener ya. El violeta es **dependiente** del rojo y el azul.

Analogía geométrica: Cada vector independiente aporta **una dirección nueva** en el espacio. Si un vector apunta a un lugar al que ya se podía llegar combinando los otros, **no suma ninguna dirección nueva**: es dependiente.

Y al revés: un vector es **independiente** si **no se puede fabricar** con los demás. Aporta algo genuinamente nuevo.

Columnas dependientes: la redundancia de características

Miren las columnas C1, C2 y C3:

$$C3 \text{ (m}^2 \text{ totales)} = C1 \text{ (m}^2 \text{ cubiertos)} + C2 \text{ (m}^2 \text{ jardín)}$$

Verifiquemos: Casa A: $80 + 20 = 100$ ✓. Casa B: $120 + 30 = 150$ ✓. Casa C: $60 + 0 = 60$ ✓.

La **columna C3 es linealmente dependiente de C1 y C2**. Es nuestra “pintura violeta”. Este fenómeno se llama **redundancia de características** o, en estadística, **multicolinealidad**.

Otro ejemplo típico en datasets reales: tener la superficie en metros cuadrados y en pies cuadrados. Una columna es la otra multiplicada por 10.76. Mismísima información, disfrazada.

¿Por qué esto es un problema para la IA?

- **No aporta información nueva.** El modelo recibe 7 columnas, pero en realidad hay menos “ideas” distintas. Es como leer dos veces el mismo párrafo de un libro creyendo que leíste dos párrafos.
- **Confunde al modelo al repartir la importancia.** Supongamos que el precio depende del tamaño. El modelo tiene que asignar un peso a C_1 , C_2 y C_3 . Pero como $C_3 = C_1 + C_2$, **hay infinitas combinaciones de pesos que dan exactamente la misma predicción**: puede darle todo el peso a C_3 , o todo a C_1 y C_2 , o cualquier mezcla. El modelo no tiene forma de decidir, y matemáticamente el problema **deja de tener una única solución**.
- **Genera inestabilidad.** Cuando las columnas son *casi* dependientes (muy parecidas, aunque no idénticas), pequeños cambios en los datos pueden hacer que los pesos salten a valores enormes y absurdos, como +500 para una columna y -499 para otra. El modelo se vuelve frágil.
- **Arruina la interpretabilidad.** Si queremos saber “¿cuánto influye el tamaño del jardín en el precio?”, mirar el peso de C_2 no sirve, porque esa influencia está mezclada con C_3 .
- **Desperdicia recursos.** Más columnas implican más cálculos, más memoria y más tiempo de entrenamiento, sin ninguna ganancia.

¿Qué hacemos? Eliminar la columna redundante (por ejemplo, quedarnos con C_1 y C_2 y descartar C_3), o usar técnicas que “comprimen” las columnas en direcciones independientes, como el **PCA** (Análisis de Componentes Principales), que verán más adelante en la materia.

Filas dependientes: ejemplos redundantes

Ahora miren las filas A y D:

$$\text{Casa A} = [80, 20, 100, 2, 1, 10, 5] \quad \text{Casa D} = [80, 20, 100, 2, 1, 10, 5]$$

¡**Son idénticas!** La fila D es igual a $1 \times$ fila A. Por lo tanto, **las filas son linealmente dependientes**. Puede ser un error de carga (la misma casa ingresada dos veces) o dos casas genuinamente iguales, pero para el modelo el efecto es el mismo.

La dependencia entre filas también puede ser más sutil: una fila que es **combinación de otras**, por ejemplo una “casa promedio” que alguien fabricó promediando las casas A y B. Tampoco aporta un caso genuinamente nuevo.

¿Cómo afecta al aprendizaje?

- **Falsa sensación de cantidad.** Creemos tener 4 ejemplos, pero en realidad tenemos 3 casas distintas. Con datasets reales, pueden creer que tienen 100.000 ejemplos y tener 60.000 únicos.
- **Sesgo en el aprendizaje.** El modelo ve la Casa A **dos veces**, así que le da **el doble de importancia**. Aprende a ajustarse especialmente bien a ese tipo de casa y descuida las demás. Es como estudiar para un examen repasando diez veces el mismo ejercicio: te sale perfecto ese, pero no aprendiste el resto.
- **Evaluaciones engañosas (fuga de datos).** Si la Casa A queda en los datos de entrenamiento y su gemela D en los de prueba, el modelo “aprueba” el examen porque ya había visto la respuesta. Creemos que el modelo es excelente, y en el mundo real falla.

- **Menos información para resolver el problema.** Cada fila independiente es como una **pista nueva** que ayuda al modelo a encontrar los pesos correctos. Una fila dependiente es una pista repetida: no ayuda a descartar ninguna posibilidad.

¿Qué hacemos? Detectar y eliminar duplicados es uno de los primeros pasos de la **limpieza de datos**, una tarea que ocupa buena parte del trabajo real de un científico de datos.

Un descubrimiento importante: el límite de las 4 filas

Acá hay un dato que sorprende a casi todos los que recién empiezan. Como cada columna es un vector de solo 4 números (vive en un espacio de 4 dimensiones), **es imposible tener más de 4 columnas independientes**. Así como en un plano (2D) no pueden tener más de 2 direcciones realmente distintas, en 4D no pueden tener más de 4.

Y en nuestro caso es peor: como la casa D repite a la A, solo hay **3 filas independientes**, así que de nuestras 7 columnas **como mucho 3 son realmente independientes**. Las otras 4 se pueden fabricar a partir de ellas.

A este número, la cantidad de direcciones verdaderamente independientes, se lo llama **rango** de la matriz. Nuestra matriz de 4×7 tiene rango 3.

¿Qué significa esto para la IA? Que **cuando tenemos más características que ejemplos, el modelo tiene más “perillas” para ajustar que pistas para decidir cómo ajustarlas**. Entonces puede memorizar los datos de entrenamiento de mil maneras distintas sin aprender nada general. Este problema se conoce como **sobreajuste** (*overfitting*), y es una de las razones por las que la IA necesita **muchos datos**: para que haya suficientes filas independientes que “atrapen” la solución correcta.

¿Qué significa que una fila o columna sea “independiente”?

Matemáticamente: un vector es independiente del resto cuando **no se puede obtener combinando los otros**. Geométricamente, **aporta una dirección nueva** en el espacio, una dirección a la que no se llegaba antes.

Para la IA:

- **Una columna independiente** es una característica que **aporta información genuinamente nueva**. Por ejemplo, la antigüedad (C6) no se deduce de los metros cuadrados ni de la distancia al centro: le cuenta al modelo algo que no sabía. Estas son las columnas valiosas.
- **Una fila independiente** es un ejemplo que **representa una situación genuinamente nueva**. Le enseña al modelo un caso que no podía deducir de los anteriores. Cuantas más filas independientes y variadas, mejor generaliza el modelo.

Resumen en una tabla:

	Independiente	Dependiente
Columnas (features)	Cada característica aporta información nueva	Características redundantes: multicolinealidad, pesos inestables
Filas (ejemplos)	Cada ejemplo enseña algo nuevo	Datos duplicados: sesgo, falsa cantidad, evaluaciones engañosas

Cierre de la clase

Hagamos un repaso del viaje de hoy:

- **Vectores:** la forma en que la IA describe cualquier cosa (una casa, un usuario, una palabra) como un punto en el espacio. Cosas parecidas quedan cerca.
- **Matrices:** tablas de vectores. Como dataset, las filas son ejemplos y las columnas son características.
- **Transformaciones lineales:** las matrices también son máquinas que estiran y rotan el espacio. Cada capa de una red neuronal es una transformación $W \cdot X$ seguida de una activación, y **entrenar es encontrar la matriz W correcta**.
- **Dependencia lineal:** detectar redundancias en columnas y filas es clave para tener datos limpios, modelos estables y evaluaciones honestas.

Mensaje final: La próxima vez que usen una IA, recuerden que detrás de esa respuesta “inteligente” hay vectores viajando por espacios de miles de dimensiones, transformados capa por capa por matrices que alguien entrenó con paciencia. Ya no es magia: **ahora saben cómo funciona**.

Ejercicio para la próxima clase: Piensen en un dataset de su interés (estudiantes, canciones, jugadores de fútbol) con al menos 5 columnas. Intenten identificar si alguna columna podría ser redundante con otras, y qué tipo de filas duplicadas podrían aparecer en la vida real.

¡Nos vemos la próxima clase, donde vamos a explorar el **producto punto** y cómo la IA mide la similitud entre vectores!

PARTE II

Cálculo para la IA: cómo aprenden las máquinas

Bienvenida a la segunda parte

En la Parte I descubrimos que la IA representa el mundo con **vectores y matrices** y que entrenar un modelo consiste en **ajustar los números de una matriz de pesos**. Pero quedó una pregunta enorme sin responder:

¿Cómo sabe la IA hacia dónde mover esos números? ¿Cómo sabe si tiene que subir o bajar un peso, y cuánto?

La respuesta está en el **cálculo**: funciones, límites, derivadas e integrales. Si el álgebra lineal es el *lenguaje* de la IA, el cálculo es su *brújula*. Tranquilos: vamos a ir paso a paso, siempre con dibujos y ejemplos de IA. El recorrido es este:

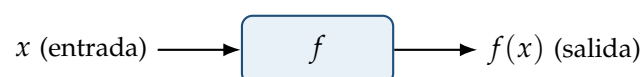
1. **Funciones**: las piezas con las que se construye un modelo.
2. **Límites y derivadas**: la brújula que le dice al modelo hacia dónde moverse.
3. **Integrales**: la herramienta para acumular y promediar errores y evaluar modelos.
4. **Estudio completo de una función de pérdida**: juntamos todo para tomar decisiones reales de entrenamiento.

6. Unidad 2: Funciones en la Inteligencia Artificial

¿Qué es una función?

Una **función** es una **regla que a cada entrada le asigna exactamente una salida**. Piénsenla como una máquina expendedora: ponen un código (entrada) y sale un producto (salida). Si ponen el mismo código dos veces, sale siempre el mismo producto.

$$f(x) = y \quad \text{“la función } f \text{ toma } x \text{ y devuelve } y\text{”}$$



¿Les suena familiar? ¡Es exactamente la idea de la “máquina” de la Parte I! Una matriz también toma un vector y devuelve otro. En IA, **todo modelo es una función**:

- Un modelo de precios es una función: **entra** una casa (un vector), **sale** un precio (un número).

- Un clasificador de imágenes es una función: **entra** una foto, **sale** la probabilidad de que sea un gato.
- ChatGPT es una función (gigantesca): **entra** un texto, **sale** la probabilidad de cada palabra siguiente.

Ejemplo: el modelo más simple del mundo

Una inmobiliaria usa la regla “el precio (en miles de dólares) es 2 veces los metros cuadrados, más 50”:

$$f(x) = 2x + 50$$

Para una casa de 100 m²: $f(100) = 2 \cdot 100 + 50 = 250$ mil dólares. El **2** y el **50** son los **parámetros** del modelo (un peso w y un sesgo b). Entrenar es encontrar los mejores valores para ellos.

Además, una red neuronal es una **composición de funciones**: la salida de una capa es la entrada de la siguiente, como una cadena de montaje:

$$\text{predicción} = f_3(f_2(f_1(x)))$$

Esta idea de “función dentro de función” va a ser clave cuando veamos la regla de la cadena.

Tres familias de funciones que van a ver todo el tiempo

- **Funciones del modelo**: transforman los datos en predicciones (por ejemplo, $f(x) = wx + b$).
- **Funciones de activación**: los “filtros no lineales” de cada neurona (ReLU, sigmoide, tangente hiperbólica).
- **Funciones de pérdida (loss)**: miden **cuánto se equivoca** el modelo. Son las protagonistas de toda esta parte, porque entrenar es, literalmente, **hacer que la pérdida sea lo más chica posible**.

Dominio: ¿qué entradas acepta la función?

Definición: Dominio

El **dominio** de una función es el **conjunto de todos los valores de entrada para los cuales la función tiene sentido** (se puede calcular).

Algunas operaciones “prohibidas” que recortan el dominio:

- **No se puede dividir por cero**: $f(x) = \frac{1}{x}$ tiene dominio $\mathbb{R} - \{0\}$ (todos los reales menos el 0).
- **No hay raíz cuadrada (real) de negativos**: $f(x) = \sqrt{x}$ tiene dominio $[0, +\infty)$.
- **El logaritmo solo acepta positivos**: $f(x) = \ln(x)$ tiene dominio $(0, +\infty)$.

El dominio en la IA: la pérdida logarítmica

En clasificación se usa mucho la **pérdida logarítmica** (*log-loss* o entropía cruzada). Si el modelo le asigna una probabilidad p a la respuesta correcta, la pérdida es:

$$L(p) = -\ln(p)$$

Como p es una probabilidad y el logaritmo no acepta el cero, el dominio útil es $(0, 1]$. **¿Qué pasa en la práctica si el modelo devuelve exactamente $p = 0$?** La computadora intenta calcular $\ln(0)$, y el entrenamiento “explota” con un error. Por eso en el código de las librerías de IA siempre verán un pequeño truco: se calcula $\ln(p + 10^{-7})$ o se “recorta” p para que nunca sea cero. **Ese truco es, literalmente, una forma de respetar el dominio.**

Imagen: ¿qué salidas puede producir la función?

Definición: Imagen (o recorrido)

La **imagen** de una función es el **conjunto de todos los valores de salida que la función puede producir**.

¿Por qué es crucial en IA? Porque la imagen de la última función de la red determina qué tipo de respuesta puede dar el modelo. Es una decisión de diseño:

Función de salida	Imagen	¿Para qué problema sirve?
Lineal: $f(x) = x$	$(-\infty, +\infty)$	Regresión: predecir precios, temperaturas (cualquier número).
Sigmoide: $\sigma(x)$	$(0, 1)$	Clasificación binaria: la salida se interpreta como probabilidad (“¿es spam?”).
ReLU: $\max(0, x)$	$[0, +\infty)$	Capas ocultas; o salidas que nunca pueden ser negativas (cantidad de ventas).
Tangente hiperbólica: $\tanh(x)$	$(-1, 1)$	Capas ocultas; salidas centradas en cero (por ejemplo, un sentimiento de -1 a 1).

Error clásico de principiante

Usar una salida **sigmoide** para predecir precios de casas. Como su imagen es $(0, 1)$, **el modelo jamás podrá predecir 250 mil dólares**, por más que entrenen durante años. No es un problema de datos ni de entrenamiento: es un problema de **imagen**.

Continuidad: dibujar sin levantar el lápiz

Definición intuitiva: Continuidad

Una función es **continua** si su gráfico se puede dibujar **sin levantar el lápiz del papel**: no tiene saltos, agujeros ni “cortes”. En la Unidad 3 vamos a dar una definición más precisa usando límites.

La interpretación que más nos importa en IA: **en una función continua, un cambio pequeño en la entrada produce un cambio pequeño en la salida**. No hay sorpresas bruscas.

Por qué la continuidad cambió la historia de la IA

Las primeras neuronas artificiales (el **perceptrón**, 1958) usaban la **función escalón**:

$$\text{escalón}(x) = \begin{cases} 0 & \text{si } x < 0 \\ 1 & \text{si } x \geq 0 \end{cases}$$

Esta función tiene un **salto** en $x = 0$: es **discontinua**. ¿El problema? Si movemos un peso un poquito, la salida o no cambia en absoluto o salta de golpe de 0 a 1. Es como intentar afinar una guitarra con una perilla que solo tiene dos posiciones: imposible ajustar “de a poquito”. La solución fue reemplazarla por la **sigmoide**, que es como un escalón “suavizado” y continuo. Con ella, cada pequeño ajuste de pesos produce un pequeño cambio en la salida, y el modelo puede **aprender gradualmente**. Esta idea, junto con las derivadas que veremos enseguida, hizo posible el entrenamiento de redes neuronales modernas.

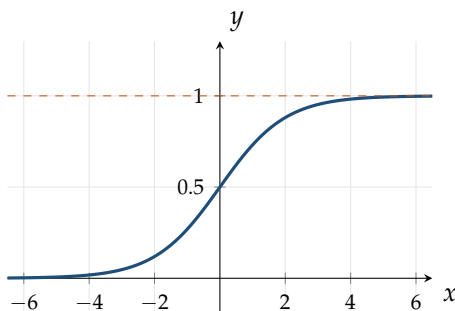
Comportamiento gráfico: leer una función con los ojos

Mirar el gráfico de una función nos dice muchísimo sin hacer cuentas. Estas son las preguntas que siempre nos hacemos:

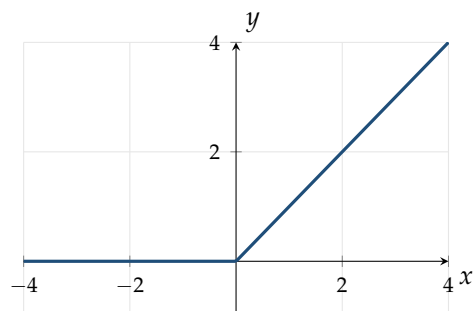
- **¿Crece o decrece?** ¿Cuando la entrada aumenta, la salida sube o baja?
- **¿Tiene asíntotas?** ¿Se acerca cada vez más a una recta sin tocarla nunca?
- **¿Se “aplana” (satura)?** ¿Hay zonas donde, por más que cambie la entrada, la salida casi no se mueve?
- **¿Tiene picos o valles?** ¿Dónde alcanza sus valores más altos o más bajos?
- **¿Tiene “esquinas” o saltos?**

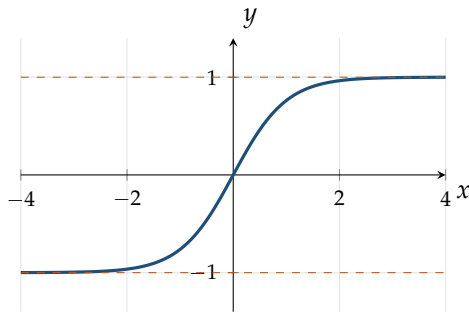
Apliquemos esto a las funciones de activación más famosas:

$$\text{Sigmoide } \sigma(x) = \frac{1}{1 + e^{-x}}$$

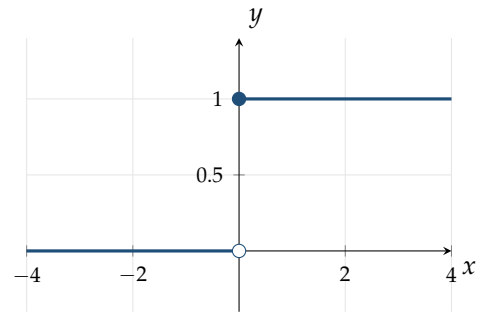


$$\text{ReLU } f(x) = \max(0, x)$$



Tangente hiperbólica $\tanh(x)$ 

Escalón (discontinua)



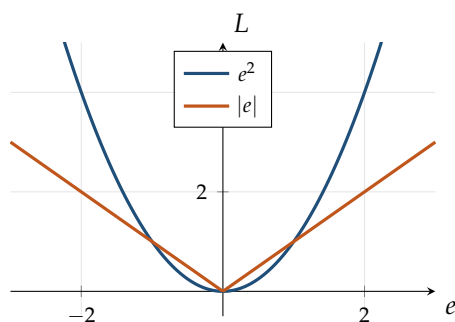
Función	Dominio	Imagen	Comportamiento gráfico
Sigmoide	$\mathbb{R}$	$(0, 1)$	Continua, siempre creciente, forma de "S". Asíntotas horizontales $y = 0$ e $y = 1$. Satura en los extremos: para x muy grande o muy chico, casi no cambia.
ReLU	$\mathbb{R}$	$[0, +\infty)$	Continua. Plana (vale 0) para negativos, recta creciente para positivos. Tiene una esquina en $x = 0$.
$\tanh$	$\mathbb{R}$	$(-1, 1)$	Continua, creciente, forma de "S" centrada en el origen. Asíntotas $y = -1$ e $y = 1$. También satura.
Escalón	$\mathbb{R}$	$\{0, 1\}$	Discontinua en $x = 0$ (salto). Plana en todo el resto.

¿Por qué nos importa la saturación? Imaginen una sigmoide en la zona plana ($x = 8$, por ejemplo). Si el modelo modifica levemente su peso, la salida prácticamente no cambia. Es como empujar una pared: por más fuerza que hagan, nada se mueve. En esas zonas **el modelo aprende lentísimo**. Este fenómeno tiene nombre, el **desvanecimiento del gradiente**, y lo vamos a cuantificar con derivadas en la próxima unidad. Es una de las razones por las que hoy ReLU es la activación más usada en capas ocultas: para positivos, **nunca se aplan**.

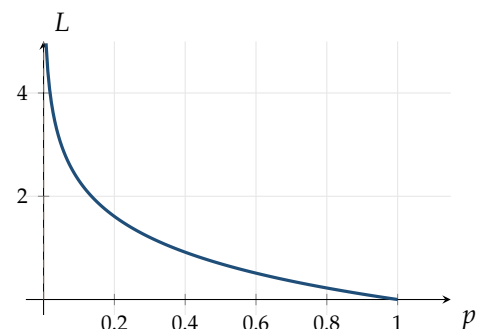
Funciones de pérdida: el "termómetro del error"

Una **función de pérdida** toma el error del modelo y lo convierte en un número: **cuanto más grande, peor**. Veamos sus gráficos, porque vamos a vivir con ellos el resto del curso. Llamemos e al error: $e = \text{predicción} - \text{valor real}$.

Cuadrática $L = e^2$ y absoluta $L = |e|$



Logarítmica $L(p) = -\ln(p)$



Leamos los gráficos como detectives:

- **Pérdida cuadrática (e^2):** es una **parábola**. Continua, suave, con un único valle en $e = 0$ (error

nulo, pérdida nula). Castiga **muchísimo** los errores grandes: un error de 3 cuesta 9, pero uno de 10 cuesta 100. Dominio $\mathbb{R}$, imagen $[0, +\infty)$.

- **Pérdida absoluta** ($|e|$): tiene forma de **V**. Castiga los errores de manera proporcional (un error del doble cuesta el doble). Es continua, pero tiene una **esquina** en $e = 0$, lo que complicará un poco las derivadas.
- **Pérdida logarítmica** ($-\ln p$): cuando el modelo asigna probabilidad $p = 1$ a la respuesta correcta, la pérdida es 0 (¡perfecto!). Pero cuando p se acerca a 0, la pérdida **crece sin límite**: hay una **asíntota vertical** en $p = 0$. Traducción: **un modelo que está muy seguro y se equivoca recibe un castigo enorme**. Es exactamente lo que queremos.

Idea para llevarse

- Un modelo de IA **es una función**, y una red neuronal es una **composición** de funciones.
- El **dominio** dice qué entradas son válidas (por eso protegemos al logaritmo del cero).
- La **imagen** dice qué respuestas puede dar el modelo (por eso elegimos la activación de salida según el problema).
- La **continuidad** permite aprender “de a poquito”.
- El **gráfico** anticipa problemas: zonas planas (aprendizaje lento), esquinas y asíntotas.

Ejercicios de la Unidad 2

1. Determinen dominio e imagen de $f(x) = x^2 + 1$ y de $g(x) = \sqrt{x - 2}$.
2. Un modelo debe predecir la probabilidad de que un cliente abandone un servicio. ¿Qué activación de salida elegirían y por qué? ¿Y si tuviera que predecir la cantidad de minutos que el cliente usará el servicio?
3. Calculen la pérdida logarítmica para $p = 0.9$, $p = 0.5$ y $p = 0.01$. ¿Qué observan sobre el castigo a los errores “confiados”?
4. Dibujen a mano la función $L(e) = e^2$ y marquen en qué zona un cambio de e produce el mayor cambio en L .

7. Unidad 3: Límites y derivadas, la brújula del aprendizaje

La metáfora que guía toda la unidad: perdidos en la montaña con niebla

Imaginen que están en una montaña, de noche y con una niebla espesa. No ven nada más allá de sus pies. Su objetivo: **llegar al punto más bajo del valle**. ¿Qué harían?

Lo más sensato es **tantear el suelo con el pie**: sentir hacia dónde baja el terreno y dar un paso en esa dirección. Repetir. Y repetir.

Eso es exactamente lo que hace una IA cuando aprende. La montaña es la **función de pérdida**. Su posición es el valor de los **pesos**. El valle es el punto de **error mínimo**. Y “tantear la inclinación del suelo” es calcular una **derivada**.

Para entender la derivada, primero necesitamos una idea previa: el límite.

Límites: ¿a qué valor se acerca?

Definición intuitiva: Límite

El **límite** de $f(x)$ cuando x tiende a a es el valor al que **se acerca** $f(x)$ cuando x se acerca cada vez más a a (sin importar qué pasa exactamente en a). Se escribe:

$$\lim_{x \rightarrow a} f(x) = L$$

La clave está en la palabra **acercarse**. El límite no pregunta “¿cuánto vale la función en ese punto?”, sino “¿hacia dónde *va* la función a medida que me acerco?”.

Límites en el infinito: el comportamiento a largo plazo

¿Qué le pasa a la sigmoide cuando x se hace enorme? Hagamos una tabla:

x	0	2	5	10	20
$\sigma(x)$	0.5	0.881	0.9933	0.99995	0.999999998

Se acerca cada vez más a 1, sin llegar nunca. Lo escribimos:

$$\lim_{x \rightarrow +\infty} \sigma(x) = 1 \quad \text{y, del otro lado,} \quad \lim_{x \rightarrow -\infty} \sigma(x) = 0$$

Estos límites son **las asíntotas horizontales** que vimos en el gráfico. Y nos garantizan algo muy útil: **la salida de la sigmoide siempre es una probabilidad válida**, por más extrema que sea la entrada.

Límites infinitos: cuando la función “explota”

Para la pérdida logarítmica:

$$\lim_{p \rightarrow 0^+} (-\ln p) = +\infty$$

(El símbolo 0^+ significa “acercándose a 0 desde la derecha”, con valores positivos). Esta es la asíntota vertical: **la pérdida no tiene techo** cuando el modelo le da probabilidad casi nula a la respuesta correcta.

Límites laterales y continuidad (ahora sí, con precisión)

A veces conviene acercarse a un punto por cada lado por separado: por la izquierda ($x \rightarrow a^-$) y por la derecha ($x \rightarrow a^+$). En la función escalón:

$$\lim_{x \rightarrow 0^-} \text{escalón}(x) = 0 \quad \text{pero} \quad \lim_{x \rightarrow 0^+} \text{escalón}(x) = 1$$

Los dos caminos llegan a lugares distintos: ahí está el **salto**. Esto nos permite dar la definición formal que prometimos:

Definición: Continuidad en un punto

f es **continua en** $x = a$ si se cumplen tres condiciones: (1) $f(a)$ existe; (2) el límite $\lim_{x \rightarrow a} f(x)$ existe (los dos límites laterales coinciden); y (3) ambos valores son iguales: $\lim_{x \rightarrow a} f(x) = f(a)$.

En palabras: “hacia donde va la función” coincide con “donde está la función”. No hay sorpresas.

Derivadas: la pendiente del terreno

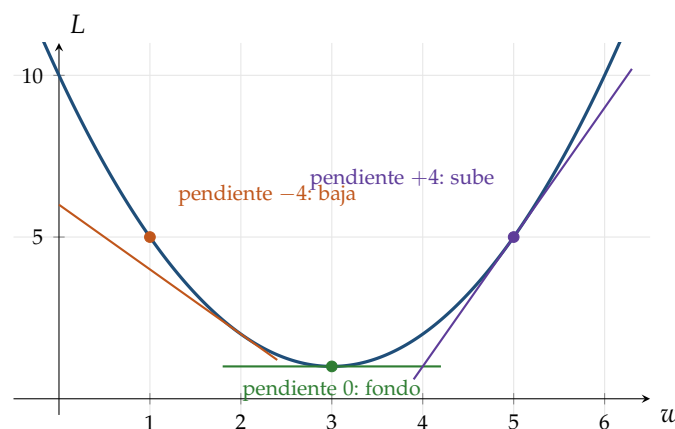
La idea: tasa de cambio

La **derivada** mide **qué tan rápido cambia una función** cuando cambiamos un poquito su entrada. Ejemplos cotidianos:

- La velocidad es la derivada de la posición: cuánto cambia la posición por cada segundo.
- En IA: la derivada de la pérdida respecto de un peso dice cuánto cambia el error si muevo ese peso un poquito.

Geométricamente, la **derivada en un punto es la pendiente de la recta tangente** a la curva en ese punto: la recta que “roza” la curva y tiene su misma inclinación ahí. Es la “inclinación del suelo bajo nuestros pies”.

Pérdida $L(w) = (w - 3)^2 + 1$ y tres rectas tangentes



La definición: un límite de pendientes

Para calcular la pendiente entre dos puntos usamos “cuánto subió dividido cuánto avanzó”. Si tomamos un punto w y otro muy cercano $w + h$:

$$\text{pendiente aproximada} = \frac{L(w + h) - L(w)}{h}$$

Si hacemos que h sea cada vez más chiquito, los dos puntos se funden en uno y obtenemos la pendiente exacta. ¡Aparece el límite!

Definición: Derivada

$$L'(w) = \lim_{h \rightarrow 0} \frac{L(w+h) - L(w)}{h}$$

También se escribe $\frac{dL}{dw}$, que se lee “cuánto cambia L por cada cambio en w ”.

Calculemos una derivada “a mano” (una sola vez en la vida)

Sea $L(w) = w^2$. Entonces:

$$\frac{(w+h)^2 - w^2}{h} = \frac{w^2 + 2wh + h^2 - w^2}{h} = \frac{2wh + h^2}{h} = 2w + h$$

Cuando $h \rightarrow 0$, esto tiende a $2w$. Por lo tanto: $L'(w) = 2w$.

En $w = 3$ la pendiente es 6 (sube fuerte); en $w = 0$ es 0 (fondo del valle); en $w = -1$ es -2 (baja). ¡Por suerte no hace falta repetir este proceso cada vez: existen reglas!

Las reglas que van a usar el 95% del tiempo

Regla	Función $\rightarrow$ Derivada	Ejemplo
Constante	$c \rightarrow 0$	$5 \rightarrow 0$ (lo que no cambia, no tiene pendiente)
Potencia	$w^n \rightarrow n w^{n-1}$	$w^3 \rightarrow 3w^2$
Múltiplo	$c f(w) \rightarrow c f'(w)$	$7w^2 \rightarrow 14w$
Suma	$f + g \rightarrow f' + g'$	$w^2 + 4w \rightarrow 2w + 4$
Exponencial	$e^w \rightarrow e^w$	(¡es su propia derivada!)
Logaritmo	$\ln w \rightarrow \frac{1}{w}$	$-\ln p \rightarrow -\frac{1}{p}$
Cadena	$f(g(w)) \rightarrow f'(g(w)) \cdot g'(w)$	$(3w-1)^2 \rightarrow 2(3w-1) \cdot 3$

La regla de la cadena: el secreto del *backpropagation*

Recuerden que una red neuronal es una **composición de funciones**. La regla de la cadena dice: **para derivar una función dentro de otra, se multiplican las derivadas de cada eslabón**. Es como un efecto dominó: si mover el peso cambia la neurona, y la neurona cambia la predicción, y la predicción cambia el error, el efecto total es el producto de todos esos efectos.

Ejemplo: la derivada del error de un modelo

Modelo: $\hat{y} = w \cdot x$. Pérdida cuadrática para un dato (x, y) : $L(w) = (wx - y)^2$. Hay una función “de afuera” (elevar al cuadrado) y una “de adentro” ($wx - y$). Por la regla de la cadena:

$$L'(w) = \underbrace{2(wx - y)}_{\text{derivada de afuera}} \cdot \underbrace{x}_{\text{derivada de adentro}}$$

Con $x = 2$, $y = 5$ y $w = 1$: $L'(1) = 2(2 - 5) \cdot 2 = -12$. La pendiente es negativa y grande: **conviene aumentar w , y con decisión**.

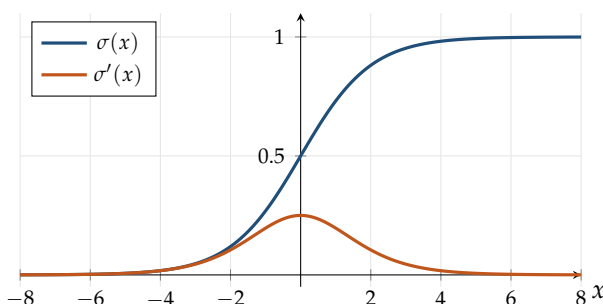
El famoso algoritmo *backpropagation* (retropropagación), con el que se entrenan *todas* las redes neuronales, no es más que **la regla de la cadena aplicada de manera ordenada, desde la última capa hacia la primera**.

La derivada de la sigmoide y el desvanecimiento del gradiente

La sigmoide tiene una derivada con una forma muy elegante:

$$\sigma'(x) = \sigma(x) (1 - \sigma(x))$$

En $x = 0$: $\sigma(0) = 0.5$, entonces $\sigma'(0) = 0.5 \cdot 0.5 = 0.25$. **Ese es su valor máximo**. En $x = 8$: $\sigma'(8) \approx 0.0003$. ¡Casi cero!



Ahora combinen esto con la regla de la cadena: en una red de 10 capas con sigmoides, la derivada total es un **producto** de 10 factores que valen, como mucho, 0.25. Y $0.25^{10} \approx 0.000001$. **La señal de aprendizaje llega prácticamente nula a las primeras capas**: esas capas no aprenden. Eso es el **desvanecimiento del gradiente**, y es la razón matemática por la que ReLU (cuya derivada vale 1 para positivos) reemplazó a la sigmoide en las capas ocultas.

Crecimiento y decrecimiento: el signo de la derivada

Esta es la relación más importante de toda la unidad:

Signo de $L'(w)$	La función...	Qué significa para el entrenamiento
$L'(w) > 0$	crece	Si aumento w , el error sube. $\Rightarrow$ Tengo que bajar w .
$L'(w) < 0$	decrece	Si aumento w , el error baja. $\Rightarrow$ Tengo que subir w .
$L'(w) = 0$	está "plana"	Estoy en un posible fondo de valle (o cima, o meseta).

¿Notan el patrón? **Siempre hay que moverse en sentido contrario al signo de la derivada**. Esa frase es el corazón del algoritmo que entrena a toda la IA moderna.

Puntos críticos y extremos

Definiciones

- Un **punto crítico** es un valor w_0 donde $L'(w_0) = 0$ (tangente horizontal) o donde la derivada no existe (por ejemplo, una esquina).
- Un **mínimo local (relativo)** es un punto más bajo que todos sus vecinos cercanos: el fondo de *un* valle.

- Un **mínimo global (absoluto)** es el punto más bajo de toda la función: el fondo del valle *más profundo*.
- Análogamente se definen el **máximo local** y el **máximo global**.

¿Cómo sabemos si un punto crítico es mínimo o máximo?

Criterio de la primera derivada (mirar el cambio de signo):

- Si L' pasa de **negativa a positiva** (baja y luego sube) $\Rightarrow$ **mínimo**. Forma de “U”.
- Si L' pasa de **positiva a negativa** (sube y luego baja) $\Rightarrow$ **máximo**. Forma de “∩”.
- Si **no cambia de signo** $\Rightarrow$ no es extremo (es una “meseta” o punto de inflexión, como w^3 en $w = 0$).

Criterio de la segunda derivada (mirar la curvatura): la segunda derivada L'' es la derivada de la derivada, y mide si la curva se dobla hacia arriba o hacia abajo.

- $L''(w_0) > 0$: la curva tiene forma de taza (“U”, **cóncava hacia arriba** o convexa) $\Rightarrow$ **mínimo**.
- $L''(w_0) < 0$: la curva tiene forma de gorro (“∩”) $\Rightarrow$ **máximo**.
- $L''(w_0) = 0$: el criterio no decide; hay que usar el de la primera derivada.

Ejemplo resuelto: la pérdida del gráfico anterior

$$L(w) = (w - 3)^2 + 1.$$

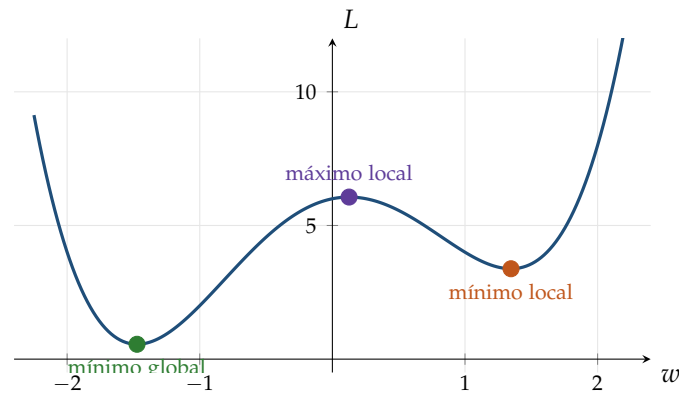
1. **Derivada** (regla de la cadena): $L'(w) = 2(w - 3)$.
2. **Punto crítico**: $2(w - 3) = 0 \Rightarrow w = 3$.
3. **Crecimiento**: si $w < 3$, $L' < 0$ (decrece); si $w > 3$, $L' > 0$ (crece). Pasa de negativa a positiva: **mínimo**.
4. **Confirmación**: $L''(w) = 2 > 0$ siempre: forma de taza. **Mínimo global**.
5. **Valor mínimo**: $L(3) = 1$.

Interpretación para la IA: el mejor peso posible es $w = 3$, y aun así el modelo tendrá una pérdida de 1 (el modelo no es perfecto, pero es lo mejor que puede hacer).

El mundo real: paisajes con muchos valles

Las funciones de pérdida de redes reales no son tazas perfectas. Miren esta función ilustrativa:

$$L(w) = w^4 - 4w^2 + w + 6 \quad \Rightarrow \quad L'(w) = 4w^3 - 8w + 1$$



Resolviendo $L'(w) = 0$ (numéricamente) hay tres puntos críticos:

w	$L(w)$	$L''(w) = 12w^2 - 8$	Tipo
-1.473	0.556	$18.04 > 0$	Mínimo global
0.126	6.063	$-7.81 < 0$	Máximo local
1.347	3.381	$13.77 > 0$	Mínimo local

La trampa de los mínimos locales

Nuestro montañista en la niebla solo siente la pendiente bajo sus pies. Si empieza en $w = 2$, bajará hasta $w \approx 1.35$ y se quedará ahí, **convencido de que llegó al fondo**, cuando el valle más profundo está en $w \approx -1.47$. Para una IA, esto significa **quedarse atascada en una solución mediocre**. Por eso en la práctica se prueban distintos puntos de partida, se usan tasas de aprendizaje variables y otras técnicas para “escapar” de valles poco profundos.

El descenso de gradiente: la derivada en acción

Ya tenemos todas las piezas. El algoritmo con el que se entrena prácticamente toda la IA es este:

Algoritmo: Descenso de gradiente

$$w_{\text{nuevo}} = w_{\text{actual}} - \eta \cdot L'(w_{\text{actual}})$$

- $L'(w)$ indica **hacia dónde sube** la pérdida; el signo menos nos hace ir **hacia donde baja**.
- η (letra griega “eta”) es la **tasa de aprendizaje** (*learning rate*): el tamaño del paso.

(Con muchos pesos, la derivada se reemplaza por el **gradiente**: un vector con la derivada respecto de cada peso. ¡Vectores otra vez!)

Ejemplo: dos montañistas con distinto tamaño de paso

Pérdida $L(w) = (w - 3)^2 + 1$, derivada $L'(w) = 2(w - 3)$. Ambos arrancan en $w_0 = 0$.

Paso	$\eta = 0.1$ (paso prudente)		$\eta = 1.1$ (paso exagerado)	
	w	$L(w)$	w	$L(w)$
0	0.000	10.00	0.00	10.00
1	0.600	6.76	6.60	13.96
2	1.080	4.69	-1.32	19.66
3	1.464	3.36	8.18	27.87
4	1.771	2.51	-3.22	39.70
5	2.017	1.97	10.47	56.73

Primer paso con $\eta = 0.1$: $L'(0) = 2(0 - 3) = -6$, entonces $w_1 = 0 - 0.1 \cdot (-6) = 0.6$. La pérdida baja en cada paso y w se acerca a 3.

Con $\eta = 1.1$ el montañista da saltos tan grandes que **pasa por encima del valle** y cae cada vez más alto del otro lado: el entrenamiento **diverge**. En la Unidad 5 vamos a calcular exactamente cuál es el límite seguro para η .

Idea para llevarse

- El **límite** describe hacia dónde va una función: asíntotas, explosiones y continuidad.
- La **derivada** es la **pendiente**: mide cuánto cambia el error al mover un peso.
- **Signo de la derivada**: positiva $\Rightarrow$ la pérdida crece; negativa $\Rightarrow$ decrece. Siempre nos movemos al revés del signo.
- Los **puntos críticos** ($L' = 0$) son candidatos a mínimos; la **segunda derivada** nos dice si es un valle.
- La **regla de la cadena** es el *backpropagation*. El **descenso de gradiente** es la derivada puesta a trabajar.

Ejercicios de la Unidad 3

1. Calculen $\lim_{x \rightarrow +\infty} \tanh(x)$ y $\lim_{x \rightarrow -\infty} \tanh(x)$ observando su gráfico. ¿Qué asíntotas tiene?
2. Deriven $L(w) = 3w^2 - 12w + 20$. Encuentren el punto crítico, determinen intervalos de crecimiento y decrecimiento, y clasifiquen el extremo.
3. Para $L(w) = (2w - 6)^2$, hagan 3 pasos de descenso de gradiente con $\eta = 0.05$ partiendo de $w_0 = 0$.
4. Expliquen con sus palabras por qué la derivada de ReLU (0 para negativos, 1 para positivos) ayuda a evitar el desvanecimiento del gradiente.

8. Unidad 4: Integrales, acumular y promediar errores

La idea: la derivada al revés

La derivada responde: “¿qué tan rápido cambia algo en este instante?”. La integral responde la pregunta opuesta: “si sé cómo cambia algo en cada instante, ¿cuánto se acumuló en total?”.

Analogía del auto: el velocímetro marca la velocidad en cada momento (eso es una derivada). Si quieren saber cuántos kilómetros recorrieron en todo el viaje, tienen que **acumular** todas esas velocidades a lo largo del tiempo. Eso es una **integral**.

En IA, la integral aparece cada vez que queremos:

- **Acumular** algo continuo: el error total a lo largo de un entrenamiento.
- **Promediar** algo continuo: la pérdida media sobre un rango de valores.
- **Medir un área** bajo una curva: como el famoso AUC, que evalúa clasificadores.

Integral indefinida: buscar la “función original”

Definición: Primitiva e integral indefinida

Una **primitiva** (o antiderivada) de f es una función F cuya derivada es f : $F'(x) = f(x)$. La **integral indefinida** reúne a todas las primitivas:

$$\int f(x) dx = F(x) + C$$

El símbolo $\int$ es una “S” estirada (de *suma*), dx indica la variable, y C es una **constante** cualquiera.

¿Por qué aparece la $+C$? Porque la derivada de una constante es 0. Las funciones x^2 , $x^2 + 5$ y $x^2 - 100$ tienen todas la misma derivada ($2x$). Al “volver hacia atrás” no podemos saber cuál era la constante original, así que la dejamos indicada.

Reglas básicas (son las de derivación, leídas al revés)

Función	Integral indefinida	Verificación (derivando)
k (constante)	$kx + C$	$(kx)' = k \checkmark$
x^n ($n \neq -1$)	$\frac{x^{n+1}}{n+1} + C$	$\left(\frac{x^{n+1}}{n+1}\right)' = x^n \checkmark$
$\frac{1}{x}$	$\ln x + C$	$(\ln x)' = \frac{1}{x} \checkmark$
e^x	$e^x + C$	$(e^x)' = e^x \checkmark$
e^{kx}	$\frac{1}{k}e^{kx} + C$	$\left(\frac{1}{k}e^{kx}\right)' = e^{kx} \checkmark$
$f + g$	$\int f + \int g$	se integra término a término

Ejemplo 1: reconstruir la pérdida a partir de su pendiente

Supongamos que solo conocemos la derivada de una pérdida: $L'(w) = 2w - 6$. ¿Cuál era la

pérdida?

$$\int (2w - 6) dw = 2 \cdot \frac{w^2}{2} - 6w + C = w^2 - 6w + C$$

Lección profunda para la IA: la derivada nos dice la *forma* del terreno (dónde está el valle: en $w = 3$), pero **no la altura** (la constante C). Por eso el descenso de gradiente puede encontrar el *mejor* peso sin saber cuánto vale la pérdida mínima: **solo necesita las pendientes**.

Ejemplo 2: una integral con exponencial

$$\int 4e^{-t/2} dt = 4 \cdot \frac{1}{-1/2} e^{-t/2} + C = -8e^{-t/2} + C$$

Verificación: $(-8e^{-t/2})' = -8 \cdot (-\frac{1}{2}) e^{-t/2} = 4e^{-t/2} \checkmark$. Vamos a usar esta integral enseguida.

Integral definida: el área bajo la curva

Definición: Integral definida y Regla de Barrow

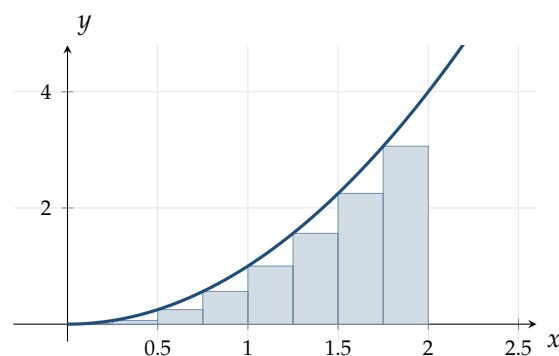
La **integral definida** $\int_a^b f(x) dx$ mide el **área (con signo) entre la curva de f y el eje x** , desde $x = a$ hasta $x = b$. Se calcula con la **Regla de Barrow** (Teorema Fundamental del Cálculo):

$$\int_a^b f(x) dx = F(b) - F(a) \quad \text{donde } F \text{ es cualquier primitiva de } f$$

(Acá la constante C se cancela sola, por eso no hace falta escribirla).

¿De dónde sale la idea del área? Imaginen que dividen el área en rectángulos muy finitos y suman sus áreas. Cuantos más rectángulos (y más finos), mejor la aproximación. La integral es el **límite** de esa suma cuando los rectángulos se vuelven infinitamente delgados. Sí: **otra vez el límite**.

$\int_0^2 x^2 dx$: área aproximada con rectángulos



Ejemplo: calcular el área exacta

$$\int_0^2 x^2 dx = \left[\frac{x^3}{3} \right]_0^2 = \frac{2^3}{3} - \frac{0^3}{3} = \frac{8}{3} \approx 2.67$$

Área “con signo”

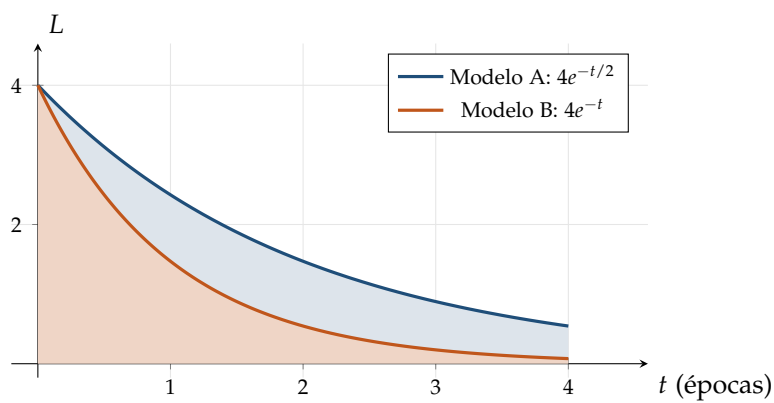
Si la curva está **debajo** del eje x , la integral cuenta esa área como **negativa**. Con las funciones de pérdida esto no nos preocupa: **las pérdidas son siempre ≥ 0** , así que su integral siempre es un área genuina.

Interpretación 1: acumulación, el costo total de un entrenamiento

Cuando entrenamos un modelo, la pérdida va bajando con el tiempo. Esa evolución se llama **curva de entrenamiento**. Supongamos que dos modelos tienen estas curvas (con t medido en épocas, es decir, pasadas completas por el dataset):

$$\text{Modelo A: } L_A(t) = 4e^{-t/2}$$

$$\text{Modelo B: } L_B(t) = 4e^{-t}$$



El **área bajo cada curva** entre $t = 0$ y $t = 4$ representa la **pérdida acumulada** durante el entrenamiento: cuánto “error total” se fue sufriendo en el camino.

$$\int_0^4 4e^{-t/2} dt = \left[-8e^{-t/2} \right]_0^4 = -8e^{-2} - (-8) = 8(1 - e^{-2}) \approx 6.92$$

$$\int_0^4 4e^{-t} dt = \left[-4e^{-t} \right]_0^4 = 4(1 - e^{-4}) \approx 3.93$$

Interpretación: ambos modelos empiezan con la misma pérdida (4), pero el modelo B acumula casi la mitad del error. **El modelo B aprende más rápido:** su curva cae antes. En la práctica, comparar áreas bajo curvas de entrenamiento es una manera de comparar la **eficiencia** de distintos algoritmos o configuraciones.

Interpretación 2: el valor promedio de una función**Definición: Valor medio de una función**

El promedio de una función f en el intervalo $[a, b]$ es:

$$\bar{f} = \frac{1}{b-a} \int_a^b f(x) dx$$

Intuición: “aplanamos” el área bajo la curva hasta convertirla en un rectángulo de la misma base. La altura de ese rectángulo es el promedio.

Conexión con algo que ya conocen: el promedio de una lista de números es “sumar y dividir por la cantidad”. La integral es la versión continua: **la integral reemplaza a la suma y la longitud del intervalo reemplaza a la cantidad.**

	Versión discreta (datos)	Versión continua
Promedio	$\frac{1}{n} \sum_{i=1}^n f(x_i)$	$\frac{1}{b-a} \int_a^b f(x) dx$

Ejemplo 1: pérdida promedio del modelo A

Durante las primeras 4 épocas:

$$\bar{L}_A = \frac{1}{4-0} \int_0^4 4e^{-t/2} dt \approx \frac{6.92}{4} \approx 1.73$$

En promedio, durante esas 4 épocas, el modelo A tuvo una pérdida de 1.73.

La pérdida media: el famoso MSE

La métrica más usada en regresión es el **Error Cuadrático Medio** (*Mean Squared Error*, MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Es, literalmente, **el promedio de las pérdidas cuadráticas de todos los datos**. Y detrás hay una idea profunda: lo que realmente nos interesa es cómo se comporta el modelo con **todos los datos posibles** del mundo real (infinitos, continuos), que se calcularía con una integral. Como solo tenemos una muestra, aproximamos esa integral con una suma. **El MSE es la versión “con datos” de un promedio integral.**

Ejemplo 2: ¿cuadrática o absoluta? La integral decide

Supongamos que los errores de un modelo están repartidos de manera uniforme entre -1 y 1 (cualquier error en ese rango es igual de probable). ¿Cuál es la pérdida promedio?

Con pérdida cuadrática:

$$\frac{1}{1 - (-1)} \int_{-1}^1 e^2 de = \frac{1}{2} \left[\frac{e^3}{3} \right]_{-1}^1 = \frac{1}{2} \left(\frac{1}{3} + \frac{1}{3} \right) = \frac{1}{3} \approx 0.33$$

Con pérdida absoluta (por simetría, el doble del lado positivo):

$$\frac{1}{2} \int_{-1}^1 |e| de = \frac{1}{2} \cdot 2 \int_0^1 e de = \left[\frac{e^2}{2} \right]_0^1 = \frac{1}{2} = 0.5$$

Interpretación: con errores chicos (menores que 1), la pérdida cuadrática es **más “indulgente”** que la absoluta, porque elevar al cuadrado un número menor que 1 lo achica. Pero con errores grandes es al revés: la cuadrática **castiga mucho más**. Esto nos ayuda a elegir: si en nuestro problema los errores grandes son graves (por ejemplo, la dosis de un medicamento), la cuadrática es la opción natural.

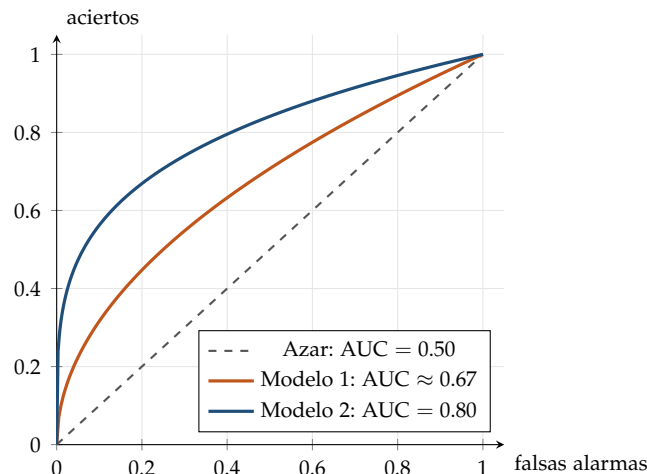
Interpretación 3: evaluar clasificadores con el área bajo la curva (AUC)

Uno de los usos más famosos de la integral en IA es la métrica **AUC** (*Area Under the Curve*). Se usa para evaluar clasificadores (“¿este email es spam?”, “¿esta radiografía muestra una enfermedad?”).

Sin entrar en detalles, un clasificador puede ser más “exigente” o más “permisivo”. La **curva ROC** muestra, para cada nivel de exigencia:

- En el eje horizontal: la **tasa de falsas alarmas** (casos negativos que el modelo marcó como positivos).
- En el eje vertical: la **tasa de aciertos** (casos positivos que el modelo detectó).

Un buen modelo detecta muchos positivos con pocas falsas alarmas: su curva **sube rápido y se pega a la esquina superior izquierda**. ¿Cómo resumir eso en un número? **Con el área bajo la curva.**



Ejemplo: comparar dos clasificadores

Supongamos (para simplificar) que las curvas ROC de dos modelos se pueden describir con fórmulas:

$$\text{Modelo 1: } f_1(x) = \sqrt{x} = x^{1/2} \quad \text{Modelo 2: } f_2(x) = x^{1/4}$$

$$\text{AUC}_1 = \int_0^1 x^{1/2} dx = \left[\frac{x^{3/2}}{3/2} \right]_0^1 = \frac{2}{3} \approx 0.67 \quad \text{AUC}_2 = \int_0^1 x^{1/4} dx = \left[\frac{x^{5/4}}{5/4} \right]_0^1 = \frac{4}{5} = 0.80$$

Un modelo que tira una moneda al aire tiene curva $f(x) = x$ y $\text{AUC} = \int_0^1 x dx = 0.5$. Un clasificador perfecto tendría $\text{AUC} = 1$.

Conclusión: el modelo 2 es mejor. Y hay una interpretación muy linda del AUC: **es la probabilidad de que, al tomar un caso positivo y uno negativo al azar, el modelo le dé mayor puntaje al positivo**. El modelo 2 “ordena bien” los casos el 80% de las veces.

Idea para llevarse

- La **integral indefinida** es la derivada al revés; la $+C$ nos recuerda que las pendientes no dicen la altura.
- La **integral definida** es un **área**, y se calcula con la Regla de Barrow: $F(b) - F(a)$.

- En IA la usamos para **acumular** (pérdida total en un entrenamiento), **promediar** (pérdida media; el MSE es su versión con datos) y **evaluar** (AUC).

Ejercicios de la Unidad 4

1. Calculen $\int (3w^2 - 4w + 1) dw$ y verifiquen derivando.
2. Calculen $\int_1^3 (w - 2)^2 dw$ e interpreten el resultado como pérdida acumulada de un peso que recorre el intervalo $[1, 3]$.
3. Una curva de entrenamiento es $L(t) = 2e^{-t}$. Calculen la pérdida promedio entre $t = 0$ y $t = 2$.
4. Un modelo tiene curva ROC $f(x) = x^{1/3}$. Calculen su AUC. ¿Es mejor o peor que el Modelo 1 del ejemplo?

9. Unidad 5: Estudio completo de una función de pérdida

Llegó el momento de juntar todo. En esta unidad vamos a hacer lo que hace un buen ingeniero de IA antes de lanzar un entrenamiento: **estudiar a fondo la función de pérdida** para tomar decisiones fundamentadas. No vamos a adivinar valores de parámetros: los vamos a **deducir**.

El problema: aprendizaje supervisado en miniatura

En el **aprendizaje supervisado** le damos al modelo ejemplos con la respuesta correcta (“esta casa costó tanto”) y le pedimos que aprenda la relación. Volvamos a nuestra inmobiliaria con un dataset diminuto:

	x : superficie (cientos de m ²)	y : precio real (cientos de miles de USD)
Casa 1	1	1
Casa 2	2	3
Casa 3	3	7

Modelo: usaremos el modelo más simple posible, con un único parámetro w (el “precio por cada 100 m²”):

$$\hat{y} = w \cdot x$$

Función de pérdida: el error cuadrático medio (MSE) que vimos en la Unidad 4:

$$L(w) = \frac{1}{3} \left[(w \cdot 1 - 1)^2 + (w \cdot 2 - 3)^2 + (w \cdot 3 - 7)^2 \right]$$

Desarrollando cada cuadrado:

$$\begin{aligned} (w - 1)^2 + (2w - 3)^2 + (3w - 7)^2 &= (w^2 - 2w + 1) + (4w^2 - 12w + 9) + (9w^2 - 42w + 49) \\ &= 14w^2 - 56w + 59 \end{aligned}$$

$$L(w) = \frac{14w^2 - 56w + 59}{3}$$

Observen algo maravilloso: tres casas, con sus números, se convirtieron en **una sola función de una variable**. Toda la información del dataset quedó “codificada” en la forma de esta curva. Ahora la estudiamos.

El estudio completo, paso a paso

Paso 1. Dominio

L es un polinomio: se puede calcular para cualquier número real. **Dominio:** $\mathbb{R}$. Traducción: el peso w puede tomar cualquier valor (positivo, negativo o cero) sin que la pérdida “se rompa”.

Paso 2. Continuidad y derivabilidad

Los polinomios son continuos y derivables en todo $\mathbb{R}$: **sin saltos, sin esquinas**. Esto garantiza que el descenso de gradiente puede usarse en cualquier punto, y que pequeños cambios en w producen pequeños cambios en la pérdida.

Paso 3. Intersecciones con los ejes

- **Con el eje vertical** ($w = 0$): $L(0) = \frac{59}{3} \approx 19.67$. Es la pérdida de un modelo que predice “precio 0” para todas las casas: el peor punto de partida razonable.
- **Con el eje horizontal** ($L = 0$): habría que resolver $14w^2 - 56w + 59 = 0$. El discriminante es $\Delta = 56^2 - 4 \cdot 14 \cdot 59 = 3136 - 3304 = -168 < 0$. **No hay soluciones reales**: la parábola nunca toca el eje.

Interpretación: no existe ningún w que haga pérdida cero. Geométricamente, los tres puntos $(1,1)$, $(2,3)$, $(3,7)$ no están alineados sobre una recta que pase por el origen, así que ningún modelo $\hat{y} = wx$ puede acertarles a todos. Esto ya es un **diagnóstico**: nuestro modelo tiene una limitación estructural.

Paso 4. Límites en el infinito

$$\lim_{w \rightarrow +\infty} L(w) = +\infty \qquad \lim_{w \rightarrow -\infty} L(w) = +\infty$$

(el término $\frac{14}{3}w^2$ domina y es positivo). No hay asíntotas. **Interpretación:** pesos extremos (en cualquier dirección) producen errores enormes. El mínimo tiene que estar en algún lugar “razonable”.

Paso 5. Derivada y puntos críticos

$$L'(w) = \frac{28w - 56}{3} = \frac{28}{3}(w - 2)$$

$$L'(w) = 0 \iff w - 2 = 0 \iff \boxed{w = 2}$$

Hay un **único punto crítico**.

Paso 6. Crecimiento y decrecimiento

Intervalo	$(-\infty, 2)$	$w = 2$	$(2, +\infty)$
Signo de $L'(w)$	−	0	+
Comportamiento de L	decrece ↘	mínimo	crece ↗
Acción del descenso de gradiente	aumentar w	quedarse	disminuir w

Paso 7. Concavidad y clasificación del extremo

$$L''(w) = \frac{28}{3} > 0 \quad \text{para todo } w$$

La función es **convexa** (forma de taza) en todo su dominio. Por lo tanto, $w = 2$ es un **mínimo global y único**. ¡No hay mínimos locales que nos atrapen! Esta es la gran ventaja de la pérdida cuadrática en modelos lineales.

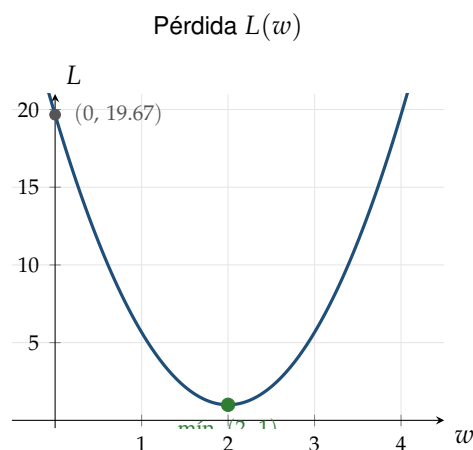
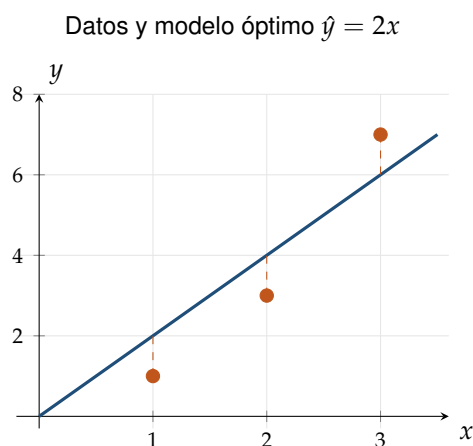
Paso 8. Valor mínimo e imagen

$$L(2) = \frac{14 \cdot 4 - 56 \cdot 2 + 59}{3} = \frac{56 - 112 + 59}{3} = \frac{3}{3} = 1$$

Imagen: $[1, +\infty)$. El modelo nunca podrá tener una pérdida menor que 1.

Verificación con los datos: con $w = 2$ las predicciones son 2, 4, 6; los errores son 1, 1, −1; los errores al cuadrado 1, 1, 1; y su promedio es exactamente 1 ✓.

Paso 9. Gráficos



En el gráfico de la izquierda, las líneas punteadas son los **errores** de cada casa: la pérdida cuadrática mide cuánto “miden” esos segmentos (al cuadrado, en promedio).

Paso 10. Integral: el valor de optimizar

¿Qué pasaría si eligiéramos w “a ojo”, al azar entre 0 y 4? La pérdida promedio sería:

$$\begin{aligned}\bar{L} &= \frac{1}{4} \int_0^4 \frac{14w^2 - 56w + 59}{3} dw = \frac{1}{12} \left[\frac{14w^3}{3} - 28w^2 + 59w \right]_0^4 \\ &= \frac{1}{12} \left(\frac{896}{3} - 448 + 236 \right) = \frac{1}{12} \cdot \frac{260}{3} \approx 7.22\end{aligned}$$

Interpretación: elegir el peso al azar nos da, en promedio, una pérdida de 7.22. Optimizar nos da 1. **Optimizar reduce el error más de 7 veces.** La integral cuantifica el valor del entrenamiento.

Ficha resumen del estudio

Aspecto	Resultado	Significado para la IA
Dominio	$\mathbb{R}$	Cualquier peso es válido.
Continuidad	Continua y derivable	Se puede usar descenso de gradiente en todo punto.
Raíces	No tiene	El modelo no puede ser perfecto con estos datos.
Límites en $\pm\infty$	$+\infty$	Pesos extremos son desastrosos.
Punto crítico	$w = 2$	Candidato a peso óptimo.
Monotonía	$\searrow$ en $(-\infty, 2)$, $\nearrow$ en $(2, +\infty)$	Indica hacia dónde mover el peso.
Concavidad	Convexa ($L'' = 28/3$)	Mínimo global único, sin trampas.
Imagen	$[1, +\infty)$	Pérdida mínima alcanzable: 1.
Promedio en $[0, 4]$	≈ 7.22	Optimizar mejora más de 7 veces.

De las matemáticas a las decisiones

Ahora usemos el estudio para responder preguntas que un equipo de IA se hace de verdad.

Decisión 1: ¿cuál es el mejor peso? (Solución analítica)

El estudio dice: $w^* = 2$. En el contexto: **el modelo aprendió que cada 100 m² suman 200 mil dólares** (2.000 USD por m²).

Si repetimos la cuenta con datos genéricos (x_i, y_i) , igualar la derivada a cero da una fórmula general:

$$w^* = \frac{\sum x_i y_i}{\sum x_i^2} \quad \text{En nuestro caso: } \frac{1 \cdot 1 + 2 \cdot 3 + 3 \cdot 7}{1 + 4 + 9} = \frac{28}{14} = 2 \checkmark$$

¿Entonces para qué sirve el descenso de gradiente? Porque en redes reales, con millones de pesos y funciones no lineales, **no existe una fórmula así**. El descenso de gradiente es el método universal. Estudiarlo en un caso simple, donde conocemos la respuesta, nos permite entender cómo configurarlo.

Decisión 2: ¿qué tasa de aprendizaje usar?

Esta es quizás la decisión más importante de todo entrenamiento. Veamos qué nos dice el estudio. La regla de actualización es:

$$w_{k+1} = w_k - \eta L'(w_k) = w_k - \eta \cdot \frac{28}{3}(w_k - 2)$$

Restemos 2 a ambos lados y llamemos $d_k = w_k - 2$ a la **distancia al óptimo**:

$$d_{k+1} = \left(1 - \frac{28}{3}\eta\right) d_k$$

¡Qué resultado tan claro! **En cada paso, la distancia al óptimo se multiplica por el factor $(1 - \frac{28}{3}\eta)$** . Para que el entrenamiento converja, ese factor tiene que ser, en valor absoluto, menor que 1:

$$\left|1 - \frac{28}{3}\eta\right| < 1 \iff 0 < \eta < \frac{6}{28} = \frac{3}{14} \approx 0.214$$

Y si el factor es exactamente 0, llegamos al óptimo **en un solo paso**: $\eta = \frac{3}{28} \approx 0.107$.

Regla general que se desprende del estudio

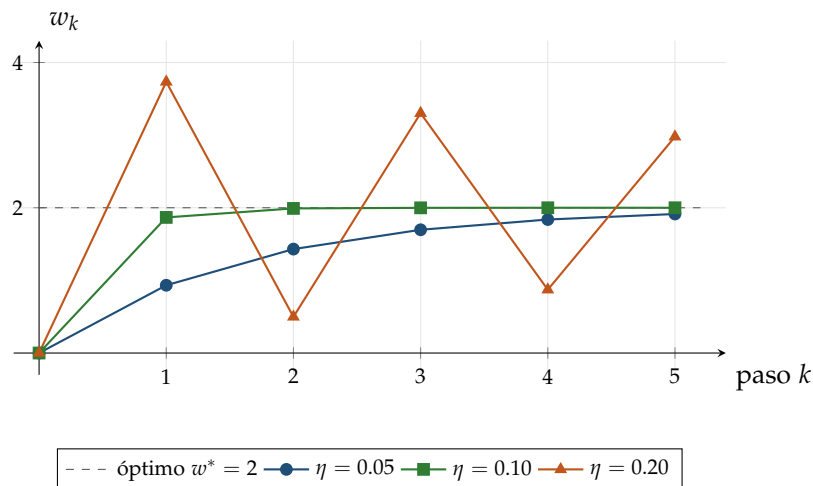
Para una pérdida cuadrática con curvatura L'' :

- El descenso de gradiente **converge** si $0 < \eta < \frac{2}{L''}$.
- El paso **ideal** es $\eta = \frac{1}{L''}$.

Intuición: cuanto más “empinada” (más curvada) es la taza, más cortos deben ser los pasos para no saltar por encima del fondo. **La segunda derivada nos dice cuán grande puede ser el paso.**

Comprobémoslo numéricamente, partiendo siempre de $w_0 = 0$:

Paso	$\eta = 0.05$ factor 0.53		$\eta = 0.10$ factor 0.07		$\eta = 0.20$ factor -0.87		$\eta = 0.25$ factor -1.33	
	w	L	w	L	w	L	w	L
0	0.000	19.67	0.000	19.67	0.000	19.67	0.00	19.67
1	0.933	6.31	1.867	1.08	3.733	15.02	4.67	34.19
2	1.431	2.51	1.991	1.0004	0.498	11.53	-1.56	60.00
3	1.697	1.43	1.999	1.0000	3.302	8.91	6.74	105.88
4	1.838	1.12	2.000	1.0000	0.872	6.94	-4.32	187.46
5	1.914	1.03	2.000	1.0000	2.978	5.46	10.43	332.48
Diagnóstico		Lento pero seguro	Casi ideal		Oscila, converge lento		Diverge	



Decisión fundamentada: elegimos $\eta \approx 0.1$. No lo adivinamos: **lo dedujimos de la segunda derivada**. En redes reales la curvatura no se conoce con exactitud, pero esta intuición explica por qué los ingenieros prueban tasas como 0.1, 0.01 y 0.001, y por qué un entrenamiento cuya pérdida “explota” se arregla **bajando** la tasa de aprendizaje.

Decisión 3: ¿importa dónde empezamos?

Como la pérdida es **convexa** (Paso 7), **cualquier punto de partida lleva al mismo mínimo**. En este problema la inicialización no es crítica. Comparen con la función de muchos valles de la Unidad 3: ahí el punto de partida decidía si terminábamos en el mínimo global o atrapados en uno local. **El estudio de la concavidad nos dice cuánto cuidado hay que tener al inicializar.**

Decisión 4: ¿el modelo es suficientemente bueno?

El estudio reveló que la pérdida mínima es 1, no 0 (Pasos 3 y 8). ¿Podemos mejorar? El problema es que nuestro modelo $\hat{y} = wx$ está obligado a pasar por el origen. Si agregamos un sesgo b (como en la Parte I):

$$\hat{y} = wx + b$$

la pérdida pasa a depender de **dos** parámetros. El procedimiento es el mismo, pero derivando respecto de cada uno por separado (derivadas parciales, que juntas forman el **gradiente**). El resultado es $w = 3$, $b = -\frac{7}{3}$, con predicciones $\frac{2}{3}, \frac{11}{3}, \frac{20}{3}$ y una pérdida mínima de:

$$L_{\min} = \frac{1}{3} \left[\left(\frac{1}{3}\right)^2 + \left(\frac{2}{3}\right)^2 + \left(\frac{1}{3}\right)^2 \right] = \frac{2}{9} \approx 0.22$$

Decisión fundamentada: agregar el sesgo reduce la pérdida mínima de 1 a 0.22 (casi 5 veces menos). El estudio de la función no solo optimiza parámetros: **también nos dice cuándo hay que cambiar el modelo.**

Decisión 5: controlar los pesos con regularización

En la Parte I vimos que, con columnas redundantes o con más características que ejemplos, hay infinitas soluciones y los pesos pueden volverse absurdos. Una solución clásica es la **regularización**: sumar a la pérdida una penalización por pesos grandes.

$$L_{\lambda}(w) = L(w) + \lambda w^2$$

Estudiémosla con la misma receta:

$$L'_{\lambda}(w) = \frac{28}{3}(w - 2) + 2\lambda w = 0 \implies w_{\lambda}^* = \frac{56}{28 + 6\lambda} \quad L''_{\lambda}(w) = \frac{28}{3} + 2\lambda > 0$$

λ	0	1	5
w_{λ}^*	2.000	1.647	0.966

Interpretación:

- A mayor λ , el peso óptimo se “encoge” hacia 0: el modelo se vuelve más prudente.
- La segunda derivada **aumenta**: la taza se vuelve más empinada y el mínimo queda más “definido”. Cuando la pérdida original tiene un fondo plano (infinitas soluciones por multico-linealidad), la regularización **lo convierte en un mínimo único**. ¡Así se conectan el álgebra lineal de la Parte I y el cálculo de la Parte II!
- El precio a pagar: con λ grande, el modelo se aleja del mejor ajuste a los datos de entrenamiento. Elegir λ es un **equilibrio** entre ajustarse a los datos y no sobreajustarse.

Protocolo general: cómo estudiar cualquier función de pérdida

Receta para llevarse

1. **Plantear** el modelo y la pérdida a partir de los datos.
2. **Dominio**: ¿qué valores de los parámetros son válidos?
3. **Continuidad y derivabilidad**: ¿se puede aplicar descenso de gradiente en todas partes?
4. **Intersecciones**: ¿se puede alcanzar pérdida cero? ¿Cuánto vale en el punto de partida?
5. **Límites**: ¿qué pasa con pesos extremos?
6. **Derivada y puntos críticos**: ¿dónde está la pendiente nula?
7. **Monotonía**: ¿hacia dónde hay que mover los pesos?
8. **Concavidad**: ¿hay un único mínimo o posibles trampas? ¿Qué tan curvada es (para elegir η)?
9. **Imagen**: ¿cuál es la mejor pérdida posible?
10. **Integrales**: ¿cuánto vale la pérdida promedio o acumulada?

11. **Decidir:** parámetros óptimos, tasa de aprendizaje, inicialización, cambios de modelo, regularización.

Idea para llevarse

Estudiar la función de pérdida es mirar el mapa antes de entrar a la montaña. Con un estudio completo pudimos encontrar el peso óptimo, deducir la tasa de aprendizaje segura, saber que la inicialización no importa, descubrir que el modelo era demasiado simple y entender qué hace la regularización. Ninguna de esas decisiones fue adivinada: **todas salieron de las matemáticas.**

Ejercicio integrador

Una empresa quiere predecir las horas de estudio (y) necesarias según la cantidad de temas de un examen (x), con los datos $(1, 2)$, $(2, 3)$, $(4, 9)$ y el modelo $\hat{y} = wx$ con pérdida MSE.

1. Planteen y desarrollen $L(w)$.
2. Realicen el estudio completo siguiendo el protocolo de 11 pasos.
3. Calculen el peso óptimo con la fórmula $w^* = \sum x_i y_i / \sum x_i^2$ y verifiquen con la derivada.
4. Determinen el intervalo de tasas de aprendizaje que garantizan convergencia y la tasa ideal.
5. Realicen 3 pasos de descenso de gradiente con la tasa ideal y con el doble de la tasa ideal. ¿Qué observan?
6. Calculen la pérdida promedio si w se elige al azar en $[0, 4]$ y compárenla con la pérdida mínima.

PARTE III

Álgebra lineal avanzada: la geometría profunda de los modelos

Bienvenida a la tercera parte

En la Parte I aprendimos a ver los datos como **vectores y matrices**. En la Parte II descubrimos cómo la IA **aprende** bajando por una función de pérdida guiada por derivadas. Ahora volvemos al álgebra lineal, pero con otros ojos: ya sabemos qué es una pendiente, una curvatura y un mínimo, y eso nos va a permitir entender conceptos que antes hubieran parecido demasiado abstractos.

La pregunta que guía esta parte: ¿qué *estructura* tienen los espacios donde viven los datos, y qué “personalidad” tienen las matrices que los transforman? Responderla explica por qué algunos modelos son estables y otros no, cómo comprimir datos sin perder información y por qué la IA moderna puede trabajar con matrices de miles de millones de números.

El recorrido:

1. **Unidad 6:** espacios vectoriales, producto punto, núcleo (y los otros “kernels” de la IA) y determinante.
2. **Unidad 7:** matrices con personalidad: simétricas, positivas, ortogonales y dispersas, y el teorema espectral que las une.

10. Unidad 6: Espacios, núcleo y determinante

Espacios vectoriales: el “universo” donde viven los datos

La idea intuitiva

Hasta ahora usamos vectores sueltos. Pero los vectores nunca viven solos: habitan un **espacio vectorial**, que es como el “universo” con sus propias reglas de juego.

Definición intuitiva: Espacio vectorial

Un **espacio vectorial** es un conjunto de objetos (vectores) en el que se pueden hacer dos operaciones **sin salirse nunca del conjunto**:

1. **Sumar** dos vectores: $\vec{u} + \vec{v}$.
 2. **Multiplicar** un vector por un número (escalar): $c \cdot \vec{v}$ (estirlo, encogerlo o darlo vuelta).
- Además, estas operaciones deben comportarse “como esperamos” (el orden de la suma no

importa, existe el vector cero, etc.). No hace falta memorizar la lista formal de reglas: la intuición es que **sumar y escalar siempre da otro habitante del mismo universo**.

El ejemplo estrella es $\mathbb{R}^n$: todas las listas de n números reales. El plano es $\mathbb{R}^2$, el espacio que habitamos es $\mathbb{R}^3$, y las casas de la Parte I (con 7 características) viven en $\mathbb{R}^7$.

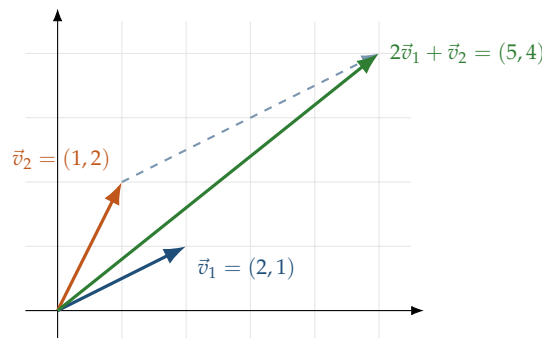
Combinaciones lineales y espacio generado

Una **combinación lineal** es cualquier mezcla de vectores usando esas dos operaciones:

$$c_1\vec{v}_1 + c_2\vec{v}_2 + \cdots + c_k\vec{v}_k$$

¡Ya la conocen! Es exactamente lo que hace una neurona: combina las características de entrada multiplicándolas por pesos y sumándolas.

El **espacio generado** (en inglés, *span*) por un grupo de vectores es el conjunto de **todos los lugares a los que se puede llegar** combinándolos. Con dos flechas que no apuntan en la misma dirección, se puede llegar a cualquier punto del plano:



Base y dimensión: los “ingredientes mínimos”

Una **base** es un conjunto de vectores que cumple dos condiciones:

- **Genera todo el espacio** (con ellos se llega a cualquier lugar).
- **Son linealmente independientes** (ninguno sobra: ¡la pintura violeta de la Parte I no entra en la base!).

La cantidad de vectores de una base se llama **dimensión** del espacio. Es la cantidad de “direcciones genuinamente distintas” que tiene. Pensar en una base es como elegir un **sistema de coordenadas**: los ejes x , y , z son la base más famosa de $\mathbb{R}^3$.

Subespacios: universos dentro del universo

Un **subespacio** es un pedazo del espacio que **también** es un espacio vectorial: si sumo o escalo vectores de ese pedazo, no me salgo de él. Ejemplos en $\mathbb{R}^3$: una recta o un plano que pasan **por el origen**.

Reconectando con la Parte I

¿Recuerdan el **rango** de la matriz de 4×7 de las casas? Era 3. Ahora podemos decirlo con más precisión: **las 7 columnas generan un subespacio de dimensión 3**. Aunque los datos parecen vivir en un espacio grande, en realidad **ocupan un subespacio mucho más chico**.

Detectar esos subespacios “escondidos” es la base de la compresión de datos (PCA), que veremos en la Unidad 7.

Espacios vectoriales en la IA: los *embeddings*

Los modelos de lenguaje convierten cada palabra en un vector (un *embedding*) de cientos de dimensiones. Lo sorprendente es que las operaciones del espacio vectorial **adquieren significado**. El ejemplo más famoso (del modelo word2vec):

$$\vec{re\acute{y}} - \vec{hombre} + \vec{mujer} \approx \vec{re\acute{ina}}$$

La diferencia $\vec{re\acute{y}} - \vec{hombre}$ captura algo así como “realeza”, y al sumársela a “mujer” se aterriza cerca de “reina”. **La IA aprendió un espacio vectorial donde las direcciones significan conceptos**. A estos espacios aprendidos se los llama **espacios latentes**, y son el corazón de ChatGPT, los generadores de imágenes y los sistemas de recomendación.

Producto punto y norma: medir en el espacio

Para comparar vectores necesitamos medir. Las dos herramientas básicas son:

Definiciones: Producto punto, norma y ángulo

- **Producto punto:** se multiplican las posiciones correspondientes y se suma todo: $\vec{a} \cdot \vec{b} = a_1b_1 + a_2b_2 + \dots + a_nb_n$.
- **Norma (longitud):** $\|\vec{a}\| = \sqrt{\vec{a} \cdot \vec{a}} = \sqrt{a_1^2 + \dots + a_n^2}$ (es el teorema de Pitágoras en n dimensiones).
- **Similitud coseno:** el coseno del ángulo entre dos vectores, $\cos \theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$. Vale 1 si apuntan igual, 0 si son perpendiculares y -1 si son opuestos.

Dos vectores son **ortogonales** (perpendiculares) cuando su producto punto es cero: no comparten ninguna dirección.

Ejemplo: ¿qué tan parecidos son Ana y Bruno? (Parte I)

Ana = [0.9, 0.1, 0.8], Bruno = [0.8, 0.2, 0.9].

$$\text{Ana} \cdot \text{Bruno} = 0.9 \cdot 0.8 + 0.1 \cdot 0.2 + 0.8 \cdot 0.9 = 0.72 + 0.02 + 0.72 = 1.46$$

$$\|\text{Ana}\| = \sqrt{0.81 + 0.01 + 0.64} = \sqrt{1.46} \approx 1.208$$

$$\|\text{Bruno}\| = \sqrt{0.64 + 0.04 + 0.81} = \sqrt{1.49} \approx 1.221$$

$$\cos \theta = \frac{1.46}{1.208 \cdot 1.221} \approx 0.99$$

Casi 1: **gustos prácticamente idénticos**. Así cuantifica un sistema de recomendación lo que en la Parte I describimos como “flechas que apuntan casi en la misma dirección”. Y noten: cada fila de $W \cdot X$ en una red neuronal es un producto punto. **Cada neurona mide cuánto se parece la entrada a su vector de pesos**.

Núcleo (kernel): lo que la matriz no puede ver

La idea: los vectores que desaparecen

Definición: Núcleo (o kernel) de una matriz

El **núcleo** de una matriz A es el conjunto de todos los vectores que A **convierte en el vector cero**:

$$\text{Nu}(A) = \ker(A) = \{ \vec{x} : A\vec{x} = \vec{0} \}$$

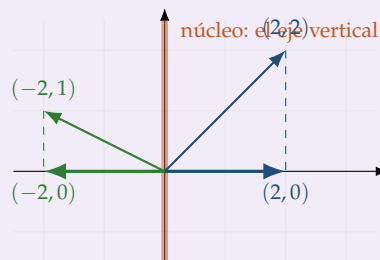
Siempre contiene al vector $\vec{0}$ y siempre es un **subespacio**.

Intuición: recuerden que una matriz es una máquina que transforma el espacio. Algunas máquinas **aplastan** ciertas direcciones hasta hacerlas desaparecer. El núcleo son esas direcciones: **lo que la transformación no puede ver**.

Ejemplo: una matriz que aplasta el plano sobre una recta

$$P = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \quad P \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x \\ 0 \end{bmatrix}$$

Esta matriz “proyecta” todo sobre el eje horizontal: borra la coordenada y . Cualquier vector vertical $(0, y)$ termina en el cero. **El núcleo es el eje vertical completo**. Es como la sombra de un objeto al mediodía: la altura se pierde.



El núcleo explica el misterio de la multicolinealidad

En la Parte I afirmamos que, con columnas redundantes, “hay infinitas combinaciones de pesos que dan exactamente la misma predicción”. **Ahora podemos demostrarlo.**

Sea X la matriz de 4×7 de las casas y $\vec{w}$ un vector de 7 pesos. Las predicciones son $X\vec{w}$. Como $C3 = C1 + C2$, el vector

$$\vec{v} = (1, 1, -1, 0, 0, 0, 0)$$

cumple $X\vec{v} = \vec{0}$ (verifiquen con la Casa A: $80 + 20 - 100 = 0$). $\vec{v}$ **está en el núcleo de X** . Entonces, para cualquier número t :

$$X(\vec{w} + t\vec{v}) = X\vec{w} + tX\vec{v} = X\vec{w} + \vec{0} = X\vec{w}$$

Podemos sumarle a los pesos cualquier múltiplo de $\vec{v}$ y **las predicciones no cambian**. Por eso el modelo no puede decidir: **toda la dirección del núcleo es invisible para la pérdida**. Geométricamente, la función de pérdida tiene un “valle plano” a lo largo de esa dirección, en lugar de un fondo único.

Teorema del rango y la nulidad

Para una matriz con n columnas:

$$\text{rango} + \dim(\text{núcleo}) = n$$

En palabras: cada columna “aporta” o bien una dirección nueva (rango), o bien una dirección invisible (núcleo). En nuestra matriz de casas: $3 + 4 = 7$. Hay **4 direcciones independientes** en las que se pueden mover los pesos sin cambiar ninguna predicción.

En una red neuronal: si una entrada cae en el núcleo de la matriz de pesos W de una capa, esa capa la convierte en cero. **La información de esas direcciones se pierde para siempre:** ninguna capa posterior puede recuperarla. Un núcleo grande es una capa “con puntos ciegos”.

Atención: la palabra “kernel” tiene tres significados en IA

Van a escuchar “kernel” todo el tiempo, y no siempre significa lo mismo. Conviene separarlos desde el principio:

Significado	Qué es	Dónde aparece
Núcleo de una matriz	Los vectores que la matriz envía al cero (lo que vimos recién).	Álgebra lineal, análisis de redundancia.
Función kernel	Una función $K(\vec{x}, \vec{z})$ que mide la similitud entre dos datos como si fuera un producto punto en un espacio más grande.	Máquinas de vectores de soporte (SVM), procesos gaussianos.
Kernel de convolución	Una matriz chiquita (por ejemplo 3×3) que se desliza sobre una imagen detectando patrones como bordes.	Redes neuronales convolucionales (visión por computadora).

Ejemplo: el “truco del kernel”

Muchas veces los datos no se pueden separar con una recta, pero sí si los llevamos a un espacio con más dimensiones. Por ejemplo, con $\vec{x} = (x_1, x_2)$ podemos crear $\varphi(\vec{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$, que vive en $\mathbb{R}^3$. El truco es que **no hace falta calcular** φ : el producto punto en el espacio grande se obtiene directamente con

$$K(\vec{x}, \vec{z}) = (\vec{x} \cdot \vec{z})^2$$

Comprobémoslo con $\vec{x} = (1, 2)$ y $\vec{z} = (3, 1)$:

- Camino largo: $\varphi(\vec{x}) = (1, 2\sqrt{2}, 4)$, $\varphi(\vec{z}) = (9, 3\sqrt{2}, 1)$, y su producto punto es $9 + 12 + 4 = 25$.
- Camino corto: $\vec{x} \cdot \vec{z} = 3 + 2 = 5$, y $5^2 = 25$. ✓

Con kernels más sofisticados, el “espacio grande” puede tener **infinitas dimensiones**, y aun así la cuenta es barata. Eso es el truco del kernel.

Ejemplo: un kernel de convolución detector de bordes

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Al apoyarlo sobre un pedazo de 3×3 píxeles, se multiplica casilla por casilla y se suma (¡un producto punto!). Si la zona es uniforme, da 0; si a la izquierda está oscuro y a la derecha claro, da un número grande. **Detecta bordes verticales.** Las redes convolucionales **aprenden** los números de sus kernels durante el entrenamiento.

Determinante: cuánto estira (o aplasta) una matriz**La idea geométrica**

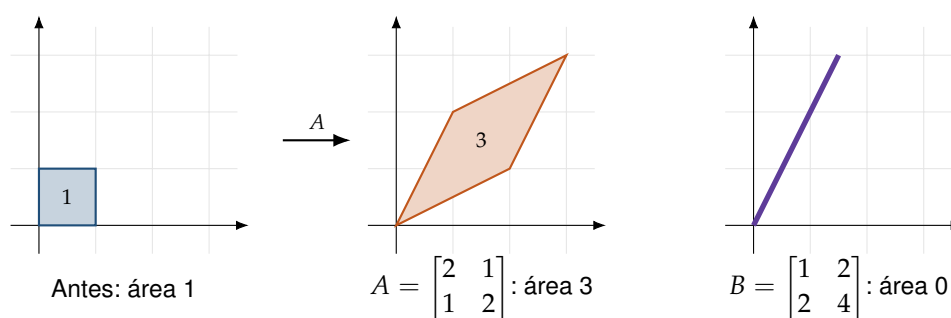
Tomen un cuadrado de lado 1 (área 1) y apliquen una matriz de 2×2 . El cuadrado se transforma en un paralelogramo. El **determinante** es el número que dice **por cuánto se multiplicó el área**.

Definición: Determinante de una matriz de 2×2

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$$

- $|\det A|$ es el **factor de cambio de área** (en 3D, de volumen; en n dimensiones, de “hipervolumen”).
- Si $\det A < 0$, la transformación además **da vuelta** el espacio (como un espejo).
- Si $\det A = 0$, la transformación **aplata** el espacio a una dimensión menor: el área se vuelve cero.

Solo se define para matrices **cuadradas**. Para matrices más grandes existen fórmulas más largas; en la práctica, lo calcula la computadora.

**Ejemplos**

- $\det \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} = 2 \cdot 1 - 0 \cdot 0 = 2$. La matriz “estirar” de la Parte I **duplica las áreas**.
- $\det \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = 0 \cdot 0 - (-1) \cdot 1 = 1$. La rotación de 90° **no cambia las áreas**: solo gira.

- $\det \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} = 4 - 1 = 3$. El cuadrado se convierte en un paralelogramo de área 3 (figura del centro).
- $\det \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} = 4 - 4 = 0$. ¡La segunda columna es el doble de la primera! Columnas dependientes $\Rightarrow$ todo el plano se aplasta sobre una recta (figura de la derecha).

El determinante como “detector” de problemas

Para una matriz cuadrada, estas afirmaciones son **todas equivalentes** (o todas verdaderas, o todas falsas):

$$\det A \neq 0 \iff \text{columnas independientes} \iff \text{núcleo} = \{\vec{0}\} \\ \iff A \text{ tiene inversa (la transformación se puede “deshacer”)}$$

¡**Todo lo que venimos viendo está conectado!** Un determinante cero significa que la matriz aplasta alguna dirección (núcleo no trivial), que hay columnas redundantes (dependencia lineal) y que la información perdida **no se puede recuperar** (no hay inversa).

En la IA:

- **Regresión lineal:** la solución clásica exacta requiere invertir la matriz $X^T X$. Si hay multicolinealidad, $\det(X^T X) = 0$ y la fórmula falla. Si el determinante es *casi* cero, la solución existe pero es **inestable**: pequeños cambios en los datos cambian mucho los pesos (lo que en la Parte I llamamos pesos “absurdos”).
- **Modelos generativos:** algunos generadores de imágenes (los *normalizing flows*) transforman ruido en imágenes paso a paso, y usan el determinante para llevar la cuenta de **cómo se estira o comprime la probabilidad** en cada transformación.
- **Distribuciones gaussianas:** el determinante de la matriz de covarianza mide cuán “dispersa” es una nube de datos en su conjunto.

Cuidado con una trampa

Un determinante chico **no siempre** indica un problema: la matriz $0.1 \cdot I$ (con I la identidad de 10×10) tiene determinante 0.1^{10} , minúsculo, pero es perfectamente estable (solo achica todo por igual). Para medir la estabilidad de verdad se usa el **número de condición**, que veremos con el teorema espectral.

Idea para llevarse

- Un **espacio vectorial** es el universo donde se puede sumar y escalar; su **dimensión** es la cantidad de direcciones independientes. Los *embeddings* son espacios vectoriales aprendidos donde las direcciones significan conceptos.
- El **producto punto** mide parecido; es la operación básica de cada neurona.
- El **núcleo** son las direcciones que una matriz no ve. Explica la multicolinealidad: moverse en el núcleo no cambia las predicciones.
- “Kernel” en IA también puede ser una función de similitud (truco del kernel) o un filtro

de convolución.

- El **determinante** mide cuánto cambia el área o el volumen. Si es cero, hay dependencia, núcleo no trivial y no hay inversa.

Ejercicios de la Unidad 6

1. ¿Los vectores $(1, 2)$ y $(3, 6)$ forman una base de $\mathbb{R}^2$? Justifiquen de dos maneras: con dependencia lineal y con el determinante.
2. Calculen la similitud coseno entre $\vec{a} = (1, 0, 1)$ y $\vec{b} = (0, 1, 0)$. Interpreten el resultado para dos usuarios de una plataforma.
3. Encuentren el núcleo de $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ y dibújenlo. ¿Qué dimensión tiene? Verifiquen el teorema del rango y la nulidad.
4. En un dataset hay una columna de superficie en m^2 y otra en pies^2 ($1 \text{ m}^2 \approx 10.76 \text{ pies}^2$). Escriban un vector que esté en el núcleo de la matriz de datos.
5. Calculen $\det \begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix}$ e interprétenlo geométricamente.

11. Unidad 7: Matrices con personalidad y el teorema espectral

Así como hay personas con rasgos de personalidad muy marcados, hay matrices con propiedades especiales que las vuelven **predecibles, estables o eficientes**. Reconocerlas es una habilidad muy valiosa: cuando un ingeniero de IA ve “esta matriz es simétrica y definida positiva”, inmediatamente sabe un montón de cosas sobre ella. Vamos a conocer a cinco de estas personalidades.

Una herramienta previa: la transpuesta

La **transpuesta** de una matriz A , que se escribe A^T , se obtiene **convirtiendo las filas en columnas** (es como “reflejar” la tabla sobre su diagonal):

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \implies A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Si A es de 2×3 , A^T es de 3×2 . En un dataset, transponer significa pasar de “filas = casas” a “filas = características”.

Matriz simétrica: la matriz “espejo”

Definición: Matriz simétrica

Una matriz cuadrada es **simétrica** si es igual a su transpuesta: $A = A^T$. Es decir, lo que está en la fila i , columna j es igual a lo que está en la fila j , columna i . La diagonal principal funciona como un espejo.

$$S = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 5 \\ 0 & 5 & 4 \end{bmatrix}$$

¿Por qué aparecen tanto en IA? Porque surgen naturalmente cada vez que medimos **relaciones mutuas**: si A se relaciona con B de cierta forma, B se relaciona con A de la misma forma.

- **Matriz de covarianza (o de correlación):** mide cómo varían juntas las características. La relación entre “m²” y “precio” es la misma que entre “precio” y “m²”.
- **Matriz de Gram** $X^T X$: contiene los productos punto entre todas las columnas del dataset. Siempre es simétrica.
- **Matriz de similitud o de kernel:** la similitud entre el dato i y el j es la misma que entre el j y el i .
- **Grafos no dirigidos:** en una red de amistades, si Ana es amiga de Bruno, Bruno es amigo de Ana.
- **Matriz Hessiana:** cuando la pérdida depende de muchos pesos, las segundas derivadas forman una matriz, la **Hessiana** H . En la fila i , columna j está “cuánto cambia la pendiente respecto del peso i cuando muevo el peso j ”. Para funciones suaves, esta matriz es simétrica. Es la generalización de L'' de la Parte II: **describe la curvatura de la pérdida en todas las direcciones**.

Matriz positiva: dos significados distintos

La expresión “matriz positiva” se usa con dos sentidos, y conviene no confundirlos.

Sentido 1: matriz con entradas positivas

Es simplemente una matriz cuyos números son todos positivos (o no negativos). Aparecen cuando las entradas son **probabilidades, conteos o pesos**:

- **Matrices de transición** (cadenas de Márkov): la probabilidad de pasar de una página web a otra. El algoritmo **PageRank**, con el que Google ordenaba sus resultados, se basa en una matriz así.
- **Matrices de atención** de los *Transformers* (la arquitectura detrás de ChatGPT): cada fila indica cuánta “atención” presta una palabra a cada una de las otras. Todas las entradas son positivas y cada fila suma 1.

Un resultado clásico (el teorema de Perron-Frobenius) garantiza que estas matrices tienen una dirección “dominante” muy estable, lo que explica, por ejemplo, que PageRank siempre converja a un ranking bien definido.

Sentido 2: matriz definida positiva (el más importante)

Definición: Matriz definida positiva

Una matriz simétrica A es **definida positiva** si para **todo** vector $\vec{x} \neq \vec{0}$:

$$\vec{x}^T A \vec{x} > 0$$

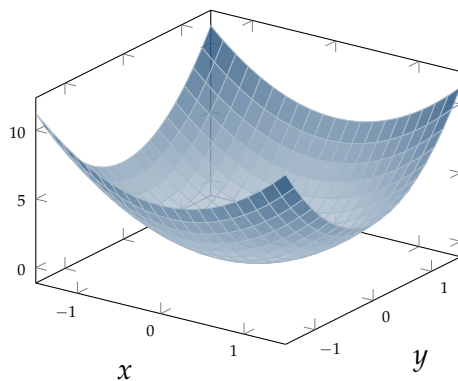
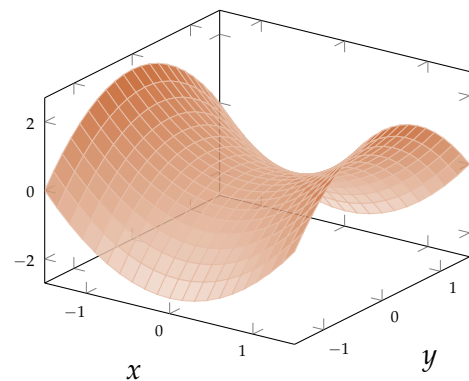
Si en lugar de $>$ vale $\geq$, se llama **semidefinida positiva**.

La expresión $\vec{x}^T A \vec{x}$ parece rara, pero es solo un número. Veamos qué representa con un ejemplo:

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}, \quad \vec{x} = \begin{bmatrix} x \\ y \end{bmatrix} \implies \vec{x}^T A \vec{x} = 2x^2 + 3y^2$$

¡Es una función de dos variables! Y como $2x^2 + 3y^2$ es positiva salvo en el origen, A es definida positiva. Su gráfico es una **taza** (un paraboloide).

La gran intuición: en la Parte II vimos que $L'' > 0$ significaba “forma de taza” y garantizaba un mínimo. “**Definida positiva**” es **exactamente la versión en muchas dimensiones de $L'' > 0$** : la función tiene forma de taza *en todas las direcciones a la vez*.

Definida positiva: $2x^2 + 3y^2$ Indefinida (silla): $x^2 - y^2$ 

A la derecha, $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ produce $x^2 - y^2$: sube en una dirección y baja en la otra. Es un **punto de silla** (como una silla de montar). No es ni mínimo ni máximo.

¿Cómo reconocer una matriz definida positiva?

- **Para 2×2 :** $\begin{bmatrix} a & b \\ b & d \end{bmatrix}$ es definida positiva si $a > 0$ y $\det = ad - b^2 > 0$.
- **En general:** si todos sus **autovalores** son positivos (lo veremos enseguida con el teorema espectral).

Dónde aparecen en IA:

- **Optimización:** si la Hessiana en un punto crítico es definida positiva, ese punto es un **mínimo**. Si es indefinida, es un **punto de silla**. En redes neuronales grandes, los puntos de silla son mucho más comunes que los mínimos locales, y son un obstáculo real para el entrenamiento.
- **Convexidad:** si la Hessiana es semidefinida positiva en todas partes, la pérdida es convexa y no hay trampas (como la pérdida cuadrática de la Unidad 5).
- **Covarianzas:** toda matriz de covarianza es semidefinida positiva (una varianza nunca es negativa).
- **Kernels válidos:** para que una función de similitud funcione como kernel, sus matrices de similitud deben ser semidefinidas positivas.

Autovectores y autovalores: las direcciones privilegiadas

Antes del teorema estrella necesitamos una idea más. Cuando una matriz transforma el espacio, la mayoría de los vectores cambian de dirección. Pero algunos vectores especiales **mantienen su dirección**: la matriz solo los estira o encoge.

Definición: Autovector y autovalor

Un vector $\vec{v} \neq \vec{0}$ es un **autovector** de A si existe un número λ ("lambda"), llamado **autovalor**, tal que:

$$A\vec{v} = \lambda \vec{v}$$

En palabras: aplicar la matriz a $\vec{v}$ es lo mismo que multiplicarlo por un número. λ dice **cuánto se estira** esa dirección.

Ejemplo: los autovectores de $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$

$$A \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix} = 3 \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad A \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 1 \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

- La dirección diagonal $(1, 1)$ se estira **al triple**: autovalor $\lambda_1 = 3$.
- La dirección $(1, -1)$ **queda igual**: autovalor $\lambda_2 = 1$.

Observen dos cosas: (1) A es simétrica, y sus dos autovectores son **perpendiculares** (su producto punto es $1 - 1 = 0$); (2) el producto de los autovalores es $3 \cdot 1 = 3 = \det A$. ¡No es casualidad!

El teorema espectral: la radiografía de una matriz simétrica

Teorema espectral (para matrices simétricas reales)

Toda matriz simétrica A cumple:

1. Todos sus autovalores son **números reales**.
2. Tiene autovectores **perpendiculares entre sí** que forman una base de todo el espacio.
3. Se puede descomponer como:

$$A = Q \Lambda Q^T$$

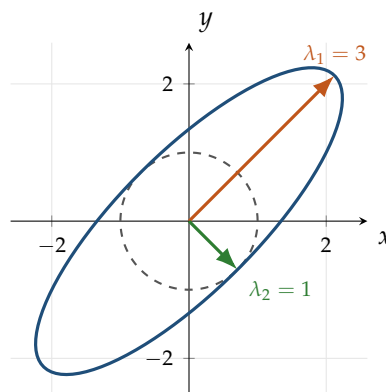
donde Q tiene como columnas los autovectores (de longitud 1) y Λ es una matriz **diagonal** con los autovalores.

La interpretación geométrica es hermosa. Toda transformación simétrica, por más complicada que parezca, es en realidad tres pasos simples:

1. Q^T : **rotar** el espacio para alinear los autovectores con los ejes.
2. Λ : **estirar** cada eje por su autovalor (sin torcer nada).
3. Q : **rotar de vuelta**.

Una matriz simétrica solo sabe estirar a lo largo de direcciones perpendiculares. Por eso transforma un círculo en una elipse cuyos ejes son los autovectores:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \text{ transforma el círculo en una elipse}$$



Ejemplo: la descomposición completa

Para $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, normalizamos los autovectores (los dividimos por su longitud $\sqrt{2}$):

$$Q = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \Lambda = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

$$Q\Lambda Q^T = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 4 & 2 \\ 2 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \checkmark$$

Consecuencias que conectan todo:

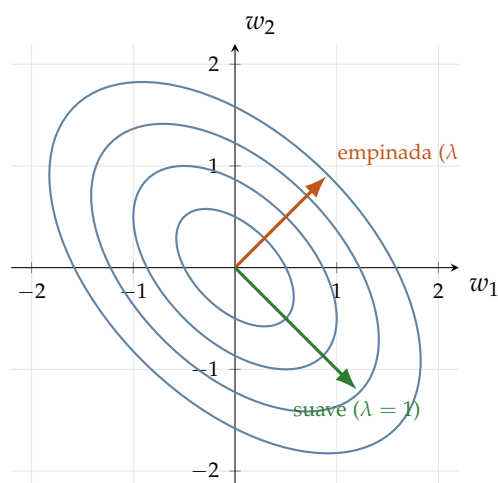
- **Determinante:** $\det A = \lambda_1 \cdot \lambda_2 \cdots \lambda_n$ (el producto de cuánto se estira cada eje es cuánto cambia el volumen).
- **Definida positiva:** A es definida positiva si y solo si **todos sus autovalores son positivos** (se estira “hacia afuera” en todas las direcciones: forma de taza).
- **Núcleo:** los autovectores con autovalor 0 forman el núcleo (direcciones que se aplastan).
- **Número de condición:** $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$. Mide cuán “desparejo” es el estiramiento. Si es enorme, la matriz está **mal condicionada**: es casi singular y los cálculos se vuelven inestables.

Aplicación 1: el descenso de gradiente en muchas dimensiones

En la Unidad 5 dedujimos que la tasa de aprendizaje segura era $\eta < 2/L''$. ¿Y con muchos pesos? Consideren la pérdida:

$$L(w_1, w_2) = w_1^2 + w_1 w_2 + w_2^2 \quad \text{con Hessiana} \quad H = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

¡Es nuestra matriz de siempre! Autovalores 3 y 1, ambos positivos: **definida positiva**, así que hay un único mínimo en $(0,0)$. El teorema espectral nos dice que la pérdida es una taza con curvatura 3 en la dirección $(1,1)$ y curvatura 1 en la dirección $(1,-1)$.

Curvas de nivel de L : ejes = autovectores

La regla de la Unidad 5 se generaliza así:

$$0 < \eta < \frac{2}{\lambda_{\max}} \quad \text{Aquí: } \eta < \frac{2}{3}$$

La **dirección más empinada limita el tamaño del paso**, pero con ese paso chico la **dirección más suave avanza lento**. Cuanto mayor es el número de condición (aquí $\kappa = 3/1 = 3$), más “alargado” es el valle y más **zigzaguea** el descenso de gradiente. En redes reales, κ puede ser de millones: por eso existen optimizadores como **Adam**, que ajustan el paso por dirección, y técnicas como la **normalización** de datos, que hacen el valle más “redondo”.

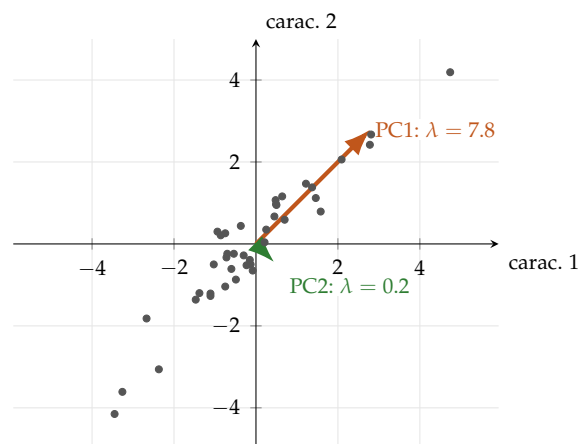
Aplicación 2: PCA, comprimir datos sin perder lo importante

En la Parte I prometimos explicar el **PCA** (Análisis de Componentes Principales). ¡Llegó el momento! Supongamos que tenemos datos de casas con dos características muy relacionadas (por ejemplo, m^2 cubiertos y m^2 totales, ya estandarizados). Su matriz de covarianza es simétrica:

$$C = \begin{bmatrix} 4 & 3.8 \\ 3.8 & 4 \end{bmatrix}$$

Por el teorema espectral, sus autovectores son $(1, 1)/\sqrt{2}$ y $(1, -1)/\sqrt{2}$, con autovalores $\lambda_1 = 7.8$ y $\lambda_2 = 0.2$.

Nube de datos y componentes principales



Interpretación:

- Cada autovalor es la **varianza** (cuánta información) en la dirección de su autovector.
- La primera dirección concentra $\frac{7.8}{7.8 + 0.2} = 97.5\%$ de la varianza total.
- Entonces podemos **reemplazar las dos columnas por una sola** (la proyección sobre PC1) perdiendo solo el 2.5% de la información. ¡Así se combate la multicolinealidad de la Parte I!
- Y noten: $\det C = 16 - 14.44 = 1.56 = 7.8 \cdot 0.2$. El autovalor chico hace que el determinante sea pequeño: la matriz está “casi” aplastada en una dirección. El número de condición es $7.8/0.2 = 39$.

En la práctica, PCA se usa con cientos de características: se calculan los autovalores de la matriz de covarianza, se ordenan de mayor a menor y **se conservan solo las direcciones que concentran la mayor parte de la varianza**. Así se comprimen imágenes, se visualizan datos de muchas dimensiones en 2D y se acelera el entrenamiento.

Matriz ortogonal: transformar sin deformar

Definición: Matriz ortogonal

Una matriz cuadrada Q es **ortogonal** si sus columnas son vectores **perpendiculares entre sí y de longitud 1**. Equivalentemente:

$$Q^T Q = I \quad \text{es decir} \quad Q^{-1} = Q^T$$

(I es la **matriz identidad**: unos en la diagonal y ceros en el resto; la matriz que “no hace nada”).

Intuición: una matriz ortogonal mueve el espacio **como un objeto rígido**: lo gira o lo refleja, pero **nunca lo estira ni lo deforma**.

Ejemplos

- **Rotación** de ángulo θ : $R_\theta = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$. Con $\theta = 90$ es la matriz $\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ de la Parte I. Verifiquemos:

$$R^T R = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I \quad \checkmark$$

- **Reflexión** (espejo sobre el eje x): $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$.
- **Permutación** (reordenar coordenadas): $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ intercambia x e y .
- La matriz Q de autovectores del teorema espectral.

Propiedades estrella:

- **Conserva longitudes y ángulos:** $\|Q\vec{x}\| = \|\vec{x}\|$. Distancias y similitudes quedan intactas.
- **Invertirla es gratis:** la inversa es simplemente la transpuesta (no hace falta ningún cálculo costoso).
- **Determinante ± 1 :** no cambia volúmenes (con -1 si incluye un espejo).
- **Número de condición 1:** son las matrices **más estables** que existen. No amplifican errores.

En la IA:

- **Inicialización ortogonal de pesos:** ¿recuerdan el desvanecimiento del gradiente? Si las matrices de pesos encogen las señales, estas se apagan capa tras capa; si las estiran, explotan. Iniciar las matrices W como ortogonales hace que las señales **conserven su tamaño** al atravesar la red.
- **Posiciones en los Transformers:** varios modelos de lenguaje modernos codifican la posición de cada palabra **rotando** su vector con matrices de rotación (la técnica RoPE). Al ser ortogonales, esas rotaciones no alteran la “información” del vector, solo su orientación.
- **PCA y cambio de base:** pasar a las componentes principales es aplicar una rotación: no se pierde ni se distorsiona nada hasta que decidimos descartar componentes.

Matriz dispersa: cuando casi todo es cero

Definición: Matriz dispersa (*sparse*)

Una matriz es **dispersa** cuando la **gran mayoría de sus entradas son cero**. Lo contrario es una matriz **densa**.

Ejemplo: la matriz de calificaciones de una plataforma

Filas = usuarios, columnas = películas, valor = calificación de 1 a 5 (0 = no la vio). Cada punto es una calificación:



De 160 casillas, solo 15 tienen datos: **más del 90% son ceros**.

Pensemos en escala real: una plataforma con 1.000.000 de usuarios y 10.000 películas tiene una matriz de 10^{10} casillas. Guardarla completa (4 bytes por número) ocuparía **40 GB**. Pero si cada usuario calificó, en promedio, 50 películas, hay solo 5×10^7 valores no nulos (0.5% de la matriz). Guardando únicamente la terna (*fila, columna, valor*) de cada uno, se necesitan unos **600 MB**. ¡Casi 70 veces menos!

Por qué las matrices dispersas son fundamentales en IA:

- **Memoria:** se guardan solo los valores no nulos y su ubicación.
- **Velocidad:** al multiplicar, se saltean los ceros (multiplicar por cero es trabajo inútil).
- **Aparecen en todas partes:**
 - Sistemas de recomendación (usuarios $\times$ productos).
 - Texto: la representación “bolsa de palabras” (documentos $\times$ vocabulario) y los vectores *one-hot* (un 1 y miles de ceros).
 - Grafos: redes sociales, mapas de rutas, moléculas (cada nodo se conecta con pocos otros).
- **Modelos dispersos a propósito:** técnicas como la regularización L1 y la **poda** (*pruning*) llevan muchos pesos a cero, generando redes más livianas que pueden correr en un celular. Algunos modelos de lenguaje enormes activan solo una pequeña parte de sus pesos para cada palabra (arquitecturas de “mezcla de expertos”).

Una sutileza importante

En la matriz de calificaciones, un cero **no significa “no le gustó”, sino “no sabemos”**. Un sistema de recomendación justamente intenta **adivinar esos huecos**. Para eso descompone la matriz en productos de matrices más chicas (factorización de matrices), aprovechando

que los gustos de las personas viven en un **subespacio de dimensión baja**: hay pocos “tipos de gusto” que combinados explican a millones de usuarios. ¡Espacios vectoriales, rango y dispersión trabajando juntos!

Resumen: la galería de personalidades

Tipo de matriz	Propiedad	Papel en la IA
Simétrica	$A = A^T$	Covarianzas, similitudes, Hessianas, grafos no dirigidos.
Entradas positivas	$a_{ij} > 0$	Probabilidades de transición, PageRank, matrices de atención.
Definida positiva	$\vec{x}^T A \vec{x} > 0$	Garantiza mínimos (Hessiana), convexidad, kernels válidos.
Ortogonal	$Q^T Q = I$	Rotaciones estables, inicialización de pesos, RoPE, PCA.
Dispersa	Casi todo es cero	Recomendación, texto, grafos, redes podadas.

Idea para llevarse

- Las matrices **simétricas** miden relaciones mutuas; el **teorema espectral** dice que solo estiran a lo largo de ejes perpendiculares: $A = Q\Lambda Q^T$.
- Definida positiva** es la versión multidimensional de $L'' > 0$: forma de taza en todas las direcciones, todos los autovalores positivos.
- Los **autovalores** de la Hessiana fijan la tasa de aprendizaje ($\eta < 2/\lambda_{\max}$); los de la covarianza dan el PCA.
- Las matrices **ortogonales** giran sin deformar: son las más estables.
- Las matrices **dispersas** permiten trabajar con datos gigantescos guardando solo lo que no es cero.

Ejercicios de la Unidad 7

- ¿Son simétricas $\begin{bmatrix} 1 & 7 \\ 7 & 3 \end{bmatrix}$ y $\begin{bmatrix} 1 & 7 \\ -7 & 3 \end{bmatrix}$? Den un ejemplo de IA con una matriz simétrica.
- Decidan si $\begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix}$ y $\begin{bmatrix} 1 & 3 \\ 3 & 1 \end{bmatrix}$ son definidas positivas usando el criterio $a > 0$ y $\det > 0$. Si fueran la Hessiana de una pérdida en un punto crítico, ¿qué tipo de punto sería cada uno?
- Verifiquen que $(1, 1)$ y $(1, -1)$ son autovectores de $\begin{bmatrix} 5 & 1 \\ 1 & 5 \end{bmatrix}$ y encuentren sus autovalores. Si fuera la Hessiana de una pérdida, ¿qué tasas de aprendizaje garantizan convergencia? ¿Cuál es el número de condición?
- Una matriz de covarianza tiene autovalores 9, 0.8 y 0.2. ¿Qué porcentaje de la varianza conserva PCA si nos quedamos con una sola componente? ¿Y con dos?
- Verifiquen que $\begin{bmatrix} 0.6 & -0.8 \\ 0.8 & 0.6 \end{bmatrix}$ es ortogonal y que conserva la longitud del vector $(3, 4)$.
- Una tienda tiene 200.000 clientes, 50.000 productos y 20 compras promedio por cliente. ¿Qué porcentaje de la matriz clientes $\times$ productos no es cero?

Cierre general del curso

Hagamos un último recorrido por todo lo que construimos, desde los ladrillos hasta el edificio completo:

Concepto matemático	Su papel en la Inteligencia Artificial
Vectores	Representan cualquier cosa (casas, usuarios, palabras) como puntos en el espacio.
Matrices	Guardan los datasets y, como transformaciones, forman las capas de las redes.
Dependencia lineal	Detecta información redundante en características y ejemplos.
Funciones	Todo modelo es una función; las activaciones y pérdidas también lo son.
Dominio e imagen	Definen qué entradas son válidas y qué respuestas puede dar el modelo.
Continuidad	Permite aprender de a poquito, con ajustes graduales.
Límites	Describen asíntotas y comportamientos extremos; son la base de derivadas e integrales.
Derivadas	Indican hacia dónde mover los pesos: son el motor del descenso de gradiente.
Regla de la cadena	Es el <i>backpropagation</i> , el algoritmo que entrena todas las redes.
Puntos críticos y extremos	Identifican los pesos óptimos y las trampas de los mínimos locales.
Segunda derivada	Revela la curvatura y fija la tasa de aprendizaje segura.
Integrales	Acumulan y promedian pérdidas, y evalúan clasificadores (AUC).
Espacios vectoriales	Son el universo de los datos y de los <i>embeddings</i> , donde las direcciones significan conceptos.
Producto punto	Mide similitud; es la operación que realiza cada neurona.
Núcleo (kernel)	Revela las direcciones invisibles para un modelo y explica la multicolinealidad.
Determinante	Detecta si una matriz aplasta el espacio y si se puede invertir.
Matrices simétricas	Representan relaciones mutuas: covarianzas, similitudes y Hessianas.
Matrices definidas positivas	Garantizan mínimos y convexidad (la versión multidimensional de $L'' > 0$).
Teorema espectral	Descompone una matriz en direcciones y estiramientos: PCA y tasa de aprendizaje.
Matrices ortogonales	Giran sin deformar: aportan estabilidad numérica y buena inicialización.
Matrices dispersas	Permiten manejar datos gigantes guardando solo lo que no es cero.

Mensaje final: Empezamos diciendo que la IA no es magia. Ahora pueden verlo con sus propios ojos: los datos son **vectores y matrices** que viven en **espacios vectoriales**; el modelo es una **función**; el error es una **función de pérdida**; aprender es **bajar por esa función guiados por la derivada**; y la **estructura de las matrices** (sus autovalores, su núcleo, su simetría) decide si ese aprendizaje será estable, rápido y eficiente. Cada vez que un modelo de IA mejora, hay una pendiente calculada, un paso dado y una matriz ajustada. Ustedes ya entienden el mecanismo. Lo que sigue es practicar, preguntar mucho y animarse a mirar dentro de la caja negra. ¡Felicitaciones por llegar hasta acá!